

Logic Programming for a Global Logistics Problem

Master's Thesis

in partial fulfillment of the requirements for
the degree of Master of Science (M.Sc.)
in Praktische Informatik

submitted by
Olcay Altay-Kern

First examiner: Prof. Dr. Matthias Thimm
Artificial Intelligence Group

Advisor: Dr. Emmanuelle Dietz
Airbus

Statement

I declare that I have written the master's thesis independently and without unauthorized use of third parties. I have only used the indicated resources and I have clearly marked the passages taken verbatim or in the sense of these resources as such. The assurance of independent work also applies to any drawings, sketches or graphical representations. The work has not previously been submitted in the same or similar form to the same or another examination authority and has not been published. By submitting the electronic version of the final version of the master's thesis, I acknowledge that it will be checked by a plagiarism detection service to check for plagiarism and that it will be stored exclusively for examination purposes.

I explicitly agree to have this thesis published on the webpage of the artificial intelligence group and endorse its public availability.

Software created for this work has been made available as open source; a corresponding link to the sources is included in this work. The same applies to any research data.

.....
(Place, Date) (Signature)

Zusammenfassung

Abstract

Logic programming has demonstrated to be a compelling alternative to imperative methodologies for addressing combinatorial optimization problems. Within an industrial setting, an Airbus R&T (Research and Technology) team has adopted ASP (Answer Set Programming) as the programming paradigm to find valid solutions for the global manufacturing setup of a future commercial aircraft. Initial research efforts have provided promising outcomes, however the obtained answer sets do not meet the expectations regarding the resulting KPIs (Key Performance Indicators). This thesis aims at implementing additional capabilities from the used framework to improve answer sets significantly with regards to the objective functions. The use of domain-heuristics and weak constraints combined with multi-shot features are expected to address significant shortcomings within the current problem encodings and lead to answer sets with improved KPIs.

Contents

1	Introduction	1
1.1	Motivation and Background	1
1.2	Problem Statement	2
1.2.1	Generalization of the Problem and Solution Space	2
1.2.2	Related Problems	8
1.2.3	Selection of the Objective Functions and Optimization State- ments	8
1.2.4	Search Tree Traversal	10
1.2.5	Thesis Hypotheses	11
1.3	Industrial System Scenarios and Constraints	11
1.4	Objectives	12
1.5	Current State of Research	16
1.6	Outline	17
2	Theoretical Background	19
2.1	Logic Programming	19
2.2	Answer Set Programming (ASP) and Stable Model Semantics	22
2.3	<i>Potassco</i> Toolset	24
2.4	Input Language of <i>clingo</i>	25
2.4.1	The Grounder <i>gringo</i>	28
2.4.2	The Solver <i>clasp</i>	28
3	Representation of the Logistics System and the Optimization Problem	31
3.1	Representation of the Logistics System	31
3.2	Issues of the initial Research Work	33
3.2.1	Optimization Statements	33
3.2.2	Usage of Aggregates	35
3.3	Overview of New Features and Expected Impact	37
3.4	Main Objective Functions	37
3.4.1	Search Tree Traversal	37
4	Methodology	39
4.1	Design of Experiments	39
4.2	Evaluation Metrics	39
4.2.1	Key Performance Indicators of Objective Functions	39
4.2.2	Variability Measurement	40
4.2.3	Search Efficiency	41
5	Results of Main Phase	42
5.1	Baseline Reference	42
5.2	Workshare Constraint	46
5.2.1	KPIs of Workshare Constraint Representation	46

5.2.2	Variability	48
5.2.3	Search Efficiency	49
5.3	Domain Heuristics	49
5.3.1	KPI evolution	49
5.3.2	Variability	50
5.3.3	Search Efficiency	50
5.4	Weak constraints	51
5.4.1	KPI evolution	51
5.4.2	Variability	51
5.4.3	Search Efficiency	51
5.5	Optimization statements	53
5.5.1	KPI evolution	53
5.5.2	Variability	53
5.5.3	Search Efficiency	55
5.6	Consolidated Result	55
6	Discussion of Main Phase	58
7	Results of Final Phase	61
7.0.1	Phase 2 Results	61
7.0.2	KPI evolution	61
7.0.3	Variability	61
7.0.4	Search Effectivity and Efficiency	61
8	Discussion of Final Phase	63
9	Conclusion	64
9.1	Summary of Contributions	64
9.2	Future Work	64

List of Figures

1	Exemplary aircraft breakdown and associated production and transportation sequence	3
2	Principle of a Multi-Echelon Production Network for an aircraft . . .	5
3	Impact of warehouses on the routing	7
4	Scatter plot of answer sets evolution of each two KPI combination. Source: [1]	8
5	Pareto optimal answer sets in red forming a Pareto Front. Source: [2]	9
6	Three answer sets with different values for the two observed objective functions f_1 and f_2 . Source: [2]	10
7	Part of a full Multi-Echelon Production Network	13
8	One selected solution with four different paths within the Multi-Echelon Production Network	14
9	Design parameters and outputs	15
10	Typical two-step architecture of ASP solutions as <i>clingo</i> , Source: [3] .	23
11	Single-Sourcing vs Multi-Sourcing: Multiple paths	36
12	Evolution of the Production Costs with respect to the Transportation Costs within the (baseline) run	44
13	Evolution of the Production Costs with respect to the CO ₂ costs within the (baseline) run	45
14	Evolution of the Transportation Costs with respect to the CO ₂ costs within the (baseline) run	45
15	Evolution of the similarity values within the baseline run.	46
16	Evolution of the KPIs for the different (workshare) implementations	46
17	Evolution of the Production Costs with respect to the Transportation Costs within the (baseline) run	47
18	Similarity evolution of two configurations	48
19	Similarity evolution of two combined configurations	48
20	Overview of the resulting KPIs from the domain heuristics runs . . .	50
21	Overview of the resulting KPIs from the weak constraint runs	52
22	Overview of the resulting KPIs from the optimization statement runs	54
23	Evolution of the Production Costs with respect to the Transportation Costs for the tested optimization statements	55
24	Evolution of the Production Costs with respect to the Transportation Costs for all tested features	57
25	Overview of the resulting KPIs from the final configuration runs . . .	62

List of Tables

1	Overview of all facts of the logic program.	32
2	Relevant rules and optimization statements of the (baseline) program excluding facts.	34
3	Overview of all new features to be tested	38
5	The number of models found for the (baseline) run	42
6	The reference KPIs of the (baseline) run.	42
7	KPI percentages of the baseline comparing the first and the last model	42
4	The reference model last (baseline) printed as a table.	43
8	The KPI percentages of each heuristic in relation to the last (baseline) model. The best results are highlighted in gray.	47
9	Similarity comparison for all (workshare) implementation options for production sites (p) and transportation similarities (t).	49
10	The number of models found for the different (workshare) implemen- tations	49
11	The KPI percentages of each heuristic in relation to the last (baseline) model. The best results are highlighted in gray.	50
12	Similarity comparison for all heuristics, with respect to the produc- tion sites (p) and transportation means (t).	51
13	The number of models found for the different heuristics	51
14	The KPI percentages of each weak constraint in relation to the last (baseline) model. The best results are highlighted in gray.	52
15	Similarity comparison for all weak constraints, with respect to the production sites (p) and transportation means (t).	52
16	The number of models found for the different weak constraints . . .	53
17	The KPI percentages of each optimization statement in relation to the last (baseline) model. The best results are highlighted in gray.	54
18	Similarity comparison for all optimization statements with respect to the production sites (p) and transportation means (t).	55
19	The number of models found for the different optimization state- ments	56
20	The KPI percentages of each final configuration in relation to the last (baseline) model. The best results are highlighted in gray.	61
21	Similarity comparison for all final configurations, with respect to the production sites (p) and transportation means (t).	61
22	The number of models found for the different weak constraints . . .	62

1 Introduction

1.1 Motivation and Background

Imagine being within an immense labyrinth of galactic scale, with only a few known honey pots. Given such vast dimensions, the likelihood of locating one within a lifetime, even with knowledge of a direct route and moving with the speed of light, is close to zero. This analogy serves as a representation of real-world challenges encountered in industrial applications, where navigating immense solution spaces with numerous constraints is commonplace. These combinatorial optimization problems are symbolized as a massive labyrinth, representing the solution space, where the honey pots symbolize the local optima and the moving speed the computational power.

In any combinatorial optimization problem characterized by such an extensive solution space, guidance is essential to even approach the targets. One would benefit from multiple starting points scattered throughout different regions of the labyrinth, ideally in promising areas. Additionally, indicators confirming whether one's directional choices are correct would be helpful. Without such aids, achieving the objective – or finding a way in the labyrinth in this metaphorical Scenario – is almost impossible.

This thesis focuses on a use case within the industrial sector, specifically concerning the development and manufacturing of a next-generation civil passenger aircraft.

In the conceptual-design stage of a commercial aircraft and its manufacturing system, industrial architects must generate and compare multiple production architectures before any landmark investment decisions are taken by the senior management. Multiple Scenarios need to be analyzed and optimized with regards to some predefined key performance indicators (KPIs) [4, 5], typically recurring production and transportation costs or non-recurring costs of investments.

The difficulty of manufacturing such a product necessitates a multitude of resources. The product design defines the manufacturing build process, which includes workloads and technologies, requiring diverse sites equipped with various expertise and resources. Moreover, in the politically charged atmosphere of the aerospace industry, numerous commitments to partners must be upheld, involving some dozens of major manufacturing sites across the world, each responsible for producing or assembling components. The main challenge is to identify the optimal configuration for a manufacturing setup that minimizes logistics efforts while keeping production cost low.

Given – for instance – the limitations of transportation means, such as the necessity of a harbor for ship use, this task represents one of many combinatorial optimization problems in industrial applications. A trivial solution – manufacturing everything at one site centrally – is unfeasible due to the substantial number of constraints as indicated above.

To address this combinatorial challenge, an Airbus central Research and Technology (R&T) team has adopted Answer Set Programming (ASP). ASP is a subclass of logic programming which belongs to the declarative programming paradigms. ASP is based on the stable model semantics and has demonstrated significant efficacy in resolving such problems. It is regarded as one of the most successful domains of artificial intelligence [6]. In this case the solver is supposed to identify valid solutions and optimize with regards to a number of different figures of merit, such as production or transportation costs. A valid solution is a solution that ensures that all required manufacturing steps are passed in the right order and that transportation - internal or external - is performed between every manufacturing step while holding the constraints. Figure 1 shows the basic steps when defining the industrial system. The product breakdown determines the manufacturing steps to be performed. The associated order of production steps is already defined. Between two production steps - in this case *Production Part 1* and *Assembly Part 1 & 2* transportation needs to be performed with the option to have warehouses as intermediate destinations. As a consequence, the design variables of the logistics system are the production locations per part, potential warehouses and the transportation means chosen between sites. A typical manufacturing and transportation path consists of repeating sequences of productions step, transport step, warehouse, transportation and the next production step. These will determine the according objective functions.

Although valid solutions have been generated, the variability of these solutions remains low, and they appear to be distant from any optimum. It is concluded that only a very small fraction of the solution space has been explored. This work is the continuation of former research work of the central R&T team of Airbus with the title *A Logic Programming approach to Global Logistics in a Co-Design Environment* [1] using the *clingo* ASP System to solve the optimization problem.

1.2 Problem Statement

The problem statement will be split into three subsections, the first one generalizing the specific problem of the thesis to known problems, the second one tackling the problem of selecting the right objective functions and according formulation of the optimization statements and the third one dealing with issues that arise with the traversal of the search space.

The detailed concepts behind the ASP features mentioned in this section will be subject to Section 2, the Theoretical Background.

1.2.1 Generalization of the Problem and Solution Space

The main problem faced in this thesis is the optimization of a *Multi-Echelon Production Network* with regards to some cost functions. A Multi-Echelon Production Network is organized in stages that must be passed in a predefined order [7].

To be more precise, this problem consists of two subproblems: First, to match parts to production sites in order to guarantee that each part is produced by at least two

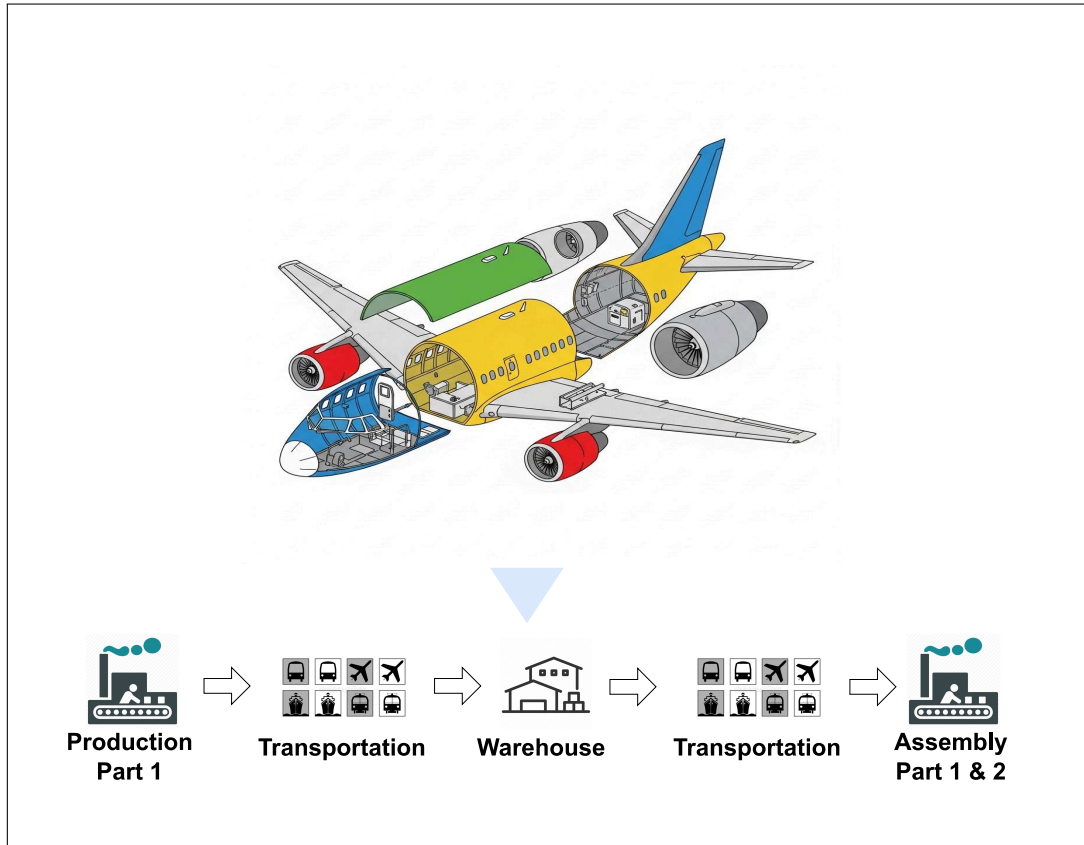


Figure 1: Exemplary aircraft breakdown and associated production and transportation sequence

sites and each site produces at least one part (or the required share by site or country), and second, to decide on the transportation routes and according means among the production sites. For better visualization of the problems, the representation as a Multi-Echelon Production Network graph is helpful, which we will do below.

The first subproblem, where parts are matched with production locations, can be regarded as a *Set Multicover Problem (SMC)*. The production locations assigned with parts to be manufactured correspond to the nodes of the Multi-Echelon Production Network. The second subproblem is about selecting transportation means among the production sites, which completes the network with its edges. Optimization inside this network, e. g. minimizing transportation costs, corresponds to the *Shortest Path in a Directed Acyclic Graph Problem*. The details are described below.

Optimal Allocation of Part Production Across Sites

A set of manufacturing sites across the world is involved in the full manufacturing chain, all with their individual capacities to produce specific components. As sites can produce multiple parts and each part need to be produced by two sites for our

specific Scenario, this leads to a many-to-many assignment problem. In this case, we deal with a special case of a Set Multicover Problem. A Set Multicover Cover Problem is to cover all elements of a universe N containing n elements at least a predefined number of times i with a minimum number of sets. It is a generalization of the *Set Cover Problem* with the main difference that elements of the universe may be covered multiple times [8].

To formalize the above problem, let there be a set $P = \{p_1, p_2, \dots, p_n\}$ of parts to be produced and a set $S = \{s_1, s_2, \dots, s_m\}$ of (production) sites. Each site $s \in S$ can produce a subset $P_s \subseteq P$ of all parts. Depending on the Sourcing Strategy, each part $p \in P$ must be produced by i_p sites (e.g. $i = 1$ for single sourcing, $i = 2$ for double sourcing etc). Each part p has a fixed workload w_p independent from the site where it is produced. And finally each site $s \in S$ has a dedicated cost factor c_s reflecting the specific manufacturing cost level of the location. Let x_{ps} be the decision variable with

$$x_{ps} = \begin{cases} 1 & \text{if part } p \text{ is produced at site } s \\ 0 & \text{otherwise} \end{cases}$$

The objective function is

$$\min \sum_{s \in S} \sum_{p \in P} c_s \cdot w_p \cdot x_{ps}$$

accordingly and we have the following constraints:

- *Workshare*: each site must produce at least one part

$$\sum_{p \in P_s} x_{ps} \geq 1 \quad \forall s \in S$$

- *Sourcing Strategy*: each part p must be produced by at least i_p sites

$$\sum_{s \in S: p \in P_s} x_{ps} \geq i_p \quad \forall p \in P$$

- *Eligibility*: parts can only be assigned to sites when they are able to produce the part

$$x_{ps} = 0 \quad \text{if } p \notin P_s$$

The size of this subproblem is determined by the given instances of production sites that can produce the number of parts to be produced. There are $n_{max} = 13$ production sites available for 34 parts to be produced. In average, every part can only be produced by $n = 10$ production sites. From these, $k = 2$ need to be selected for every part to comply with the double Sourcing Strategy (will be explained in Section 1.3). This can be calculated with the binomial coefficient with the following number of possible combinations for every part [9]:

$$\binom{n}{k} = \frac{n!}{k!(n-k)!}$$

As we have overall $P = 34$ parts to be produced, the overall number of possible combinations is

$$\left(\frac{n!}{k!(n-k)!} \right)^P = 1.6E56$$

Optimal Routing Across Production Sites The second problem is finding the shortest path in a logistics network which can be regarded as a Multi-Echelon Supply Chain and Production System. It represents an industrial setup that is structured in manufacturing stages. Every stage must be passed downstream and for all stages there might be multiple sites to fulfill the task. It can typically be represented as a *Directed Acyclic Graph* (DAG) [10]. See an example of a Multi-Echelon production system with four echelons (Elementary Part Suppliers, Sub-Assembly Sites, Assembly Sites and Final Assembly Sites) in Figure 2. All need to be passed downstream in this order to produce a product.

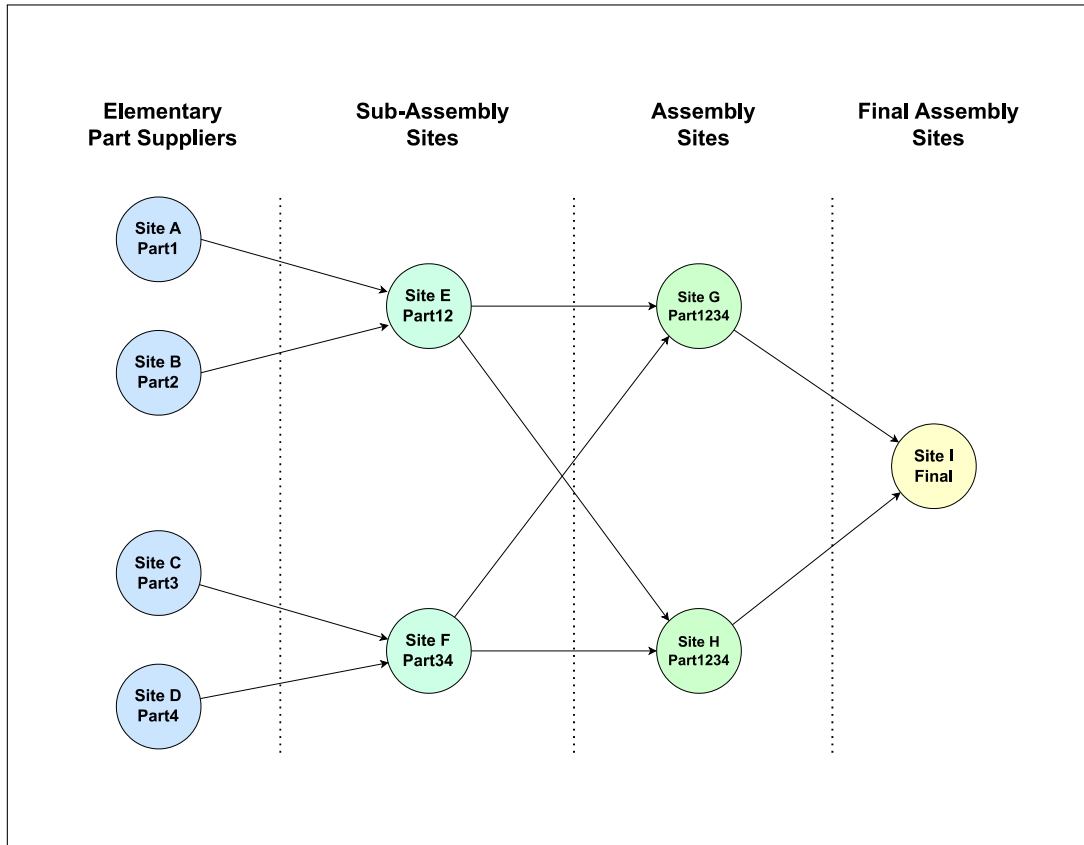


Figure 2: Principle of a Multi-Echelon Production Network for an aircraft

Common challenges in industrial applications are the optimization of such a network with regard to some KPIs [2], e. g. production or transportation costs, or analyzing the robustness of the network against disruptions .

Let G be a DAG with $G = (V, E)$ with $u \in V$ and V is partitioned into l layers

$$V = V_1 \cup V_2 \cup \dots \cup V_l$$

where each V_i (with $i \in \{1, \dots, l\}$) is the set of nodes representing the possible production sites at echelon i . Edges $E \subseteq V \times V$ are directed and can only go from one level i to the next level $i + 1$. Each edge $(u_i, v_{i+1}) \in E$ has a cost $c_{tm}(u_i, v_{i+1})$ where tm is any transportation means that is available between the nodes u_i and v_{i+1} .

As there are multiple possibilities to select production sites per echelon, binary decision variables must be introduced for any edge:

$$x_{uv} = \begin{cases} 1 & \text{if } u \text{ and } v \text{ and } edge(u, v) \text{ is selected in the path} \\ 0 & \text{otherwise} \end{cases}$$

The optimization problem with regards to any cost c_{tm} of a transportation mean tm at the edge (u_i, v_{i+1}) can then be expressed by:

$$\min \sum_{(u_i, v_{i+1}) \in E} c_{tm}(u_i, v_{i+1})$$

Problem formulations that are expressed as shortest path problems in a DAG with vertices V and edges E can be solved in linear time with $\Theta(V + E)$, for instance with a topological sort method [9].

The size of this subproblem is determined by the transport routes which are depending on the selected production sites from the first subproblem, the available transport means for these routes, the part to be transported and potential warehouses that are used in between. There are $R = 132$ core routes (the connection between two production sites that need perform consecutive manufacturing steps) to be served for a double Sourcing Strategy (see Section 1.3 for explanation of the Sourcing Strategy and see Section 3.2.2 for the explanation why a double Sourcing Strategy requires four times the routes compared to a single Sourcing Strategy), $n_1 = 17$ warehouses and 30 transportation means available. In average, every route and part can only be served by $n_2 = 10$ transport means. Between every route there is the option to have none, $k_1 = 0$ (which we call *Direct*), one, $k_1 = 1$ (called *Via 1*) or two, $k_1 = 2$ warehouses (called *Via 2*). See the warehouse Scenario explained in Section 1.3 for detailed description. Selecting a warehouse to be used between two production sites will make two routes necessary instead of a single direct route. And two warehouse will lead to three routes instead of one. See image 3 for visualization. As a consequence the number of routes can increase up to three times of the 132 routes mentioned earlier resulting in $R_{max} = 394$ possible routes maximum for

the double Sourcing Strategy.

As for the first subproblem, this can be calculated with the binomial coefficient with the following number of possible combinations split into two key decisions, i.e. the selection of the number of warehouses for a route between two production sites and the selection of a transportation mean for every resulting route to be served.

For the selection of warehouses the solution space or number of possible combinations can be calculated for all 132 core routes with:

$$\left(\binom{n_1}{k_1} + \binom{n_1}{k_2} + \binom{n_1}{k_2} \right)^R = (1 + 17 + 136)^{132} = 5.6E288$$

And for the selection of one transportation mean for each of the worst case number of routes R_{max} :

$$\binom{n_2}{1}^{R_{max}} = 10^{394} = 10E394$$

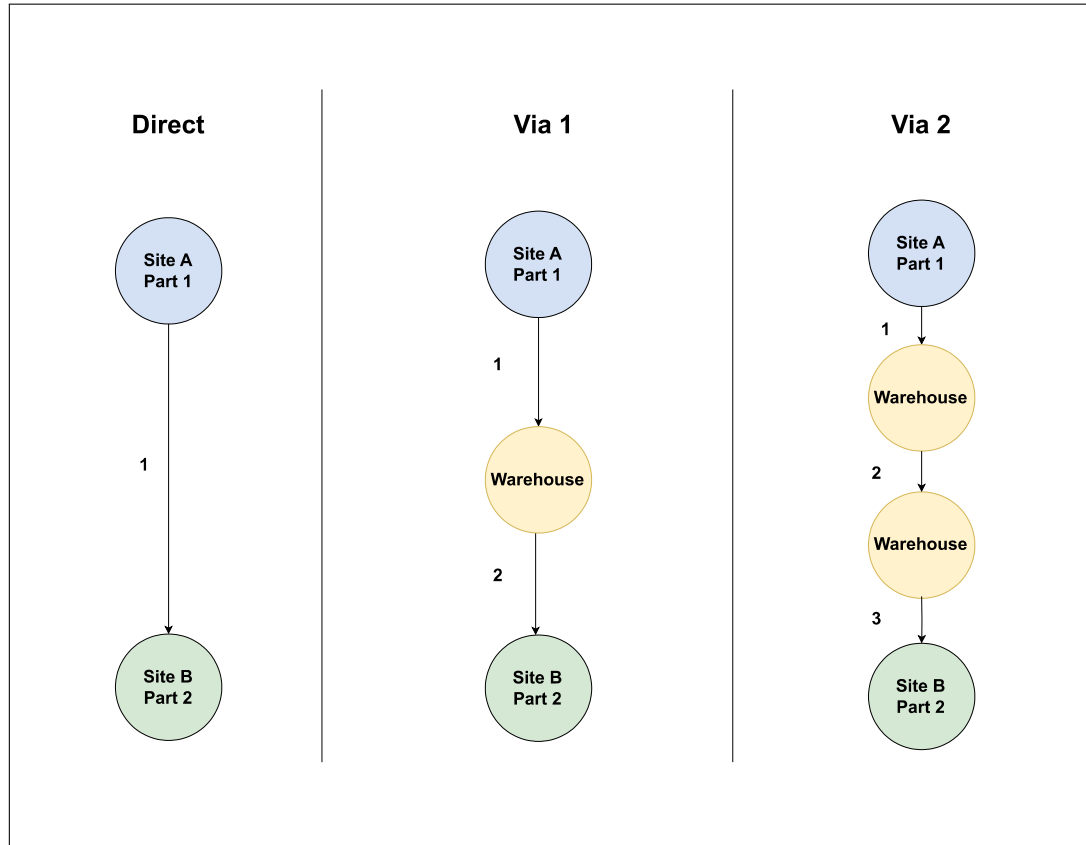


Figure 3: Impact of warehouses on the routing

Regarding the KPIs, the production costs are determined by the assignment of parts to production sites and therefore subject to the Set Multicover Problem. Transportation costs and CO₂ emissions on the other hand are subject to the routing problem as they can be represented and computed from the costs of the edges.

ASP can solve problems that are NP-hard or harder [6], enabling to solve the problem as defined here, but the size of the problem will be very challenging in terms of runtime.

1.2.2 Related Problems

Typically the problem above comes with another well-known problem, the *Multi-Commodity Flow Problem*, which appears when multiple part types flow through a logistics network. This problem additionally considers capacity and time factors of the network to be optimized making it a *Minimum-Cost-Flow-Problem* [11] which is not subject to this thesis.

Due to the obligation that every partner must hold a share in the manufacturing process, a *Traveling-Salesperson-Problem (TSP)* component is also a characteristic of the overall problem. The main problems and constraints addressed in the TSP, finding the shortest path with passing every node, are already tackled in the problem generalization by the *Set Multicover Problem* with the constraint that every site must produce a part - ensuring that every node of the DAG is passed.

1.2.3 Selection of the Objective Functions and Optimization Statements

The implementation of the initial research activity and the according answer sets are far away from an optimum with regards to a multi-objective function. An optimum is an answer set which optimizes the predefined KPIs in a multi-objective optimization formula. Figure 4 is visualizing the effects observed for a *Pareto* optimization indicating that solutions do not hit the Pareto frontier. A Pareto optimization aims at finding the best solution between two conflicting objective functions [2].

See the evolution of the KPIs in the scatter plot in Figure 4. It is not important to see the details of the axes, but rather the general evolution of the models and according KPIs.

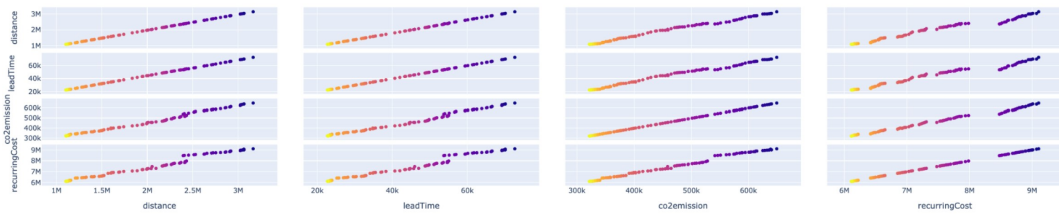


Figure 4: Scatter plot of answer sets evolution of each two KPI combination. Source: [1]

Each dot represents a solution - answer set - of the logistics system. Each subplot is showing the according value for every answer set with regard to any combination of two KPIs out of the predefined ones. The iterations evolve over time from the top right corner of the plot to improved solutions in the direction of the bottom left corner for every subplot. The single dots representing answer sets are linearly moving until the end. The process does not provide an optimum for any combination. An optimum in a Pareto diagram is normally indicated by the so-called *Pareto Front*. The set of Pareto optimal solutions in Pareto diagrams is supposed to form a front around the bottom left corner as both KPIs can not be improved simultaneously anymore [2]. The principle is shown in Figure 5.

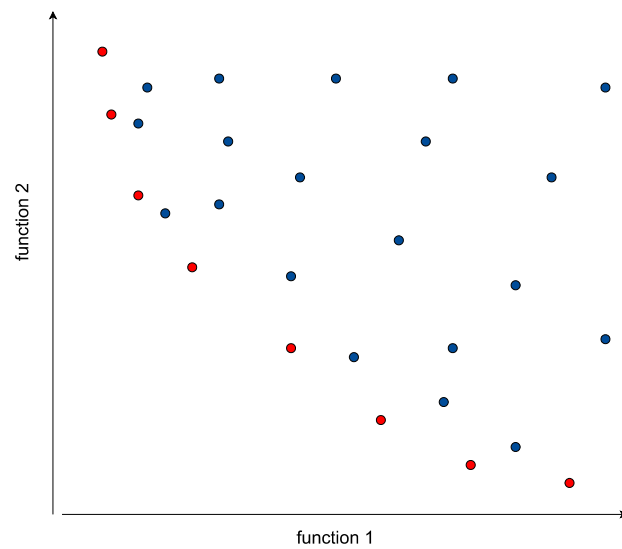


Figure 5: Pareto optimal answer sets in red forming a Pareto Front. Source: [2]

A Pareto optimal solution *dominates* another answer set if the two resulting objective function values are both better than for the compared answer set. An example is shown in Figure 6. Answer set *A* dominates answer set *B*, but does not dominate answer set *C*. Pareto optimal solutions are not dominated by any other solution.

Apparently the Pareto Front is not reached for any combination of KPIs and the given predefined maximum number of models for the optimization. It is acknowledged by the research team and confirmed by the industrial architects,

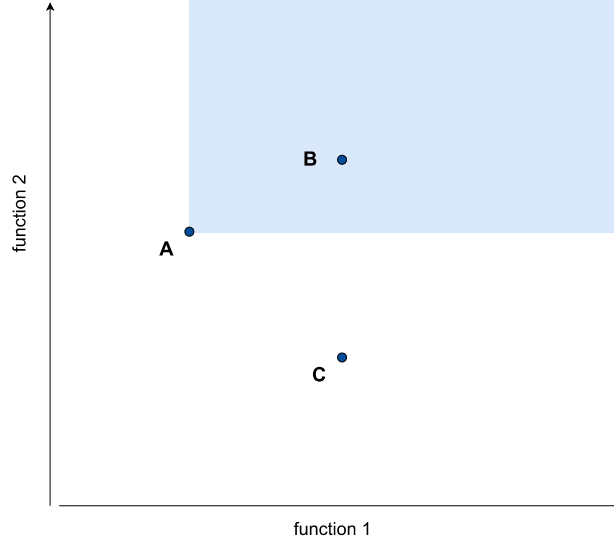


Figure 6: Three answer sets with different values for the two observed objective functions f_1 and f_2 . Source: [2]

who are the customers of the research work, that part of the chosen KPIs are not the right ones to be taken into account for architectonic trades and according decisions. The analysis of the solutions shows that variability of the selected predicates or atoms which build an answer set is low. The generated solutions cover a very small portion of the search space, which leads to the next problem: the limited traversal of the search space.

1.2.4 Search Tree Traversal

The problem encoding consists of a large number of facts and rules without negations. This leads to a positive propositional program which is *stratified* (see Section 2.1) once the program is grounded, which only takes a few seconds. This prevents from single atoms cyclic dependencies. All rules are *safe* (see Section 2.1). All together, this has a positive impact on the complexity of the program and the associated effort and run-times of the solver.

On the other hand, the absence of negations will result in the inference processes to not encounter many conflicts during solving while solvers like *clingo*, which use

Conflict-Driven-Nogood-Learning (CDNL) techniques, may not be able to prune the search space effectively. As the search space is not pruned sufficiently, the optimizer does not succeed in converging for any combination of KPIs.

The static weighting of the optimization statements does lead to a unique search space traversal, missing opportunities to actively explore promising spaces.

The conclusion of the observations above is that partially the choice of the KPIs is not suitable for a Pareto optimization due to their linear dependencies. Additionally, the selected KPIs do not serve to properly perform the trades requested to enable decisions on architectural alternatives.

On top it is assumed that only a small amount of the search space is explored, which is suggested by the low variability of answer sets and the linear direction of the answer set evolution towards the theoretical optimum. The search space is not pruned sufficiently to enable convergence of the optimizer.

1.2.5 Thesis Hypotheses

As a consequence of the observations made and described in the previous sections, the following **hypotheses** are established:

- H1 The static and independent formulation of the optimization statements does not enable a proper Pareto optimization and results in a unique convergence behavior of the optimizer missing diversity in the traversal of the search space
- H2 The problem encoding does not support main mechanism of the solver, i.e. CDNL, and the search space is therefore not properly pruned to enable to find an optimum
- H3 The solver is not actively guided through the search space and promising areas are not actively searched to increase both, variability of answer sets and the probability to find an optimum

1.3 Industrial System Scenarios and Constraints

The character of the industrial system to be optimized is strongly determined by some general strategies or assumptions concerning the supply chain and manufacturing setup which can be regarded as possible standard *Scenarios*. These Scenarios do have an impact on the topology of the industrial system that can significantly change the systems properties. They might act as additional constraints or choices, potentially having an impact on the complexity of the system and the according problem formulation. The Scenarios to be considered in this work are *Sourcing Strategy* and *Warehouse Strategy*.

The Sourcing Strategy defines whether a production step has to be performed by minimum one or multiple sites to ensure redundancy in the system which might

lead to more robustness of the industrial system. The strategy might well be defined depending and applied on the production stage of the Multi-Echelon Production Network.

Warehouses contribute to the robustness of the network, too, but can also have an impact on reachability of production sites with certain transportation means, because these warehouses provide an opportunity to switch transportation means. It can be chosen to have no warehouse, one warehouse or two warehouses between two production steps.

Two important constraints need to be implemented: one to guarantee each site produces at least one part and one to make sure that every part is produced by at least one site. The first constraint might be refined with an interval of expected workshares per site or country and is called *Workshare Constraint*.

Batching considerations will not be tackled in this thesis. Batching refers to the possibility to transport several numbers of a part with one transportation mean depending on the available space. An even more advanced approach which is applied often in practice is the concept of *mixed batching*, i.e. the possibility to transport different sorts of parts in multiple numbers with one transportation mean. Practically, batching will directly influence the number of necessary transports between two sites. In this thesis, a single-part batching is assumed, meaning that every transportation mean transports exactly one part.

1.4 Objectives

The objective of this thesis is to find solutions to the problem of identifying valid industrial and logistics setups with ASP for the next generation Airbus civil aircraft manufacturing system with Pareto optimal KPIs regarding the multi-objective functions.

A valid solution in our context is an answer set that ensures that all required manufacturing steps are passed in the right order and that transportation is performed between every manufacturing step. This finally results in the representation of a Multi-Echelon Production Network, which describes a manufacturing system that is arranged in stages or multiple layers with materials and products flowing downstream [7]. Figure 7 aims at visualizing such Multi-Echelon Production Network with different possibilities to produce specific parts (represented as nodes) and transportation links among them (represented as edges).

For the specific problem tackled in this thesis, 30 locations around the globe can act as production or warehouse sites with 34 parts to be produced. For the transportation among the selected production and warehouse sites 34 different transport means are available. The valid solutions are constrained by the facts that not every production site can produce every part, not every transportation mean is available between every site and not every part can be transported with every transportation mean.

There must be locations that produce the parts needed for a full aircraft. For our

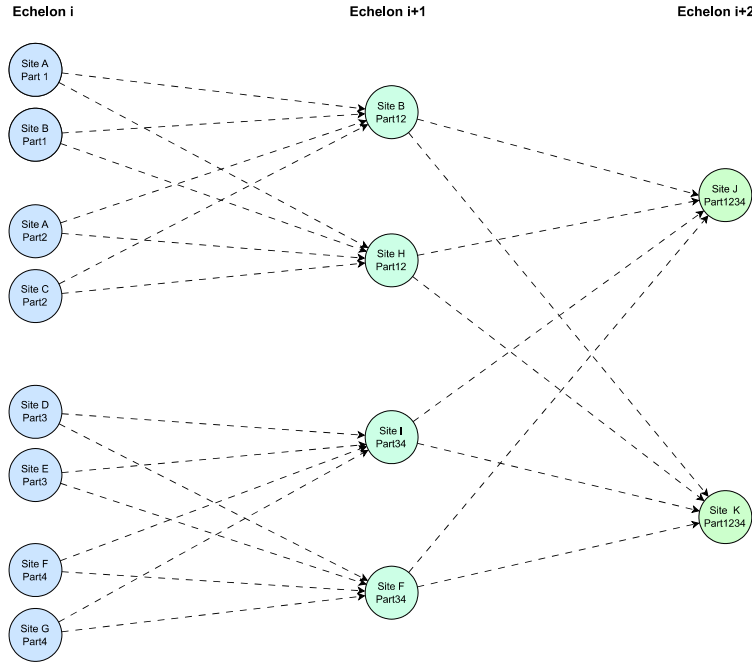


Figure 7: Part of a full Multi-Echelon Production Network

problem we need two locations to produce a specific part, which is called *Double-Sourcing*. This is a significant element of the Scenario to be considered, i.e. the Sourcing Strategy. The Sourcing Strategy is defined per production stage of the *Multi-Echelon Production Chain*.

The next major Scenario to be considered is the Warehouse Scenario, which requests that between two manufacturing steps there should be none, one or two warehouses.

Figure 7 visualizes a part of a full Multi-Echelon Production Network with three stages from echelon i to echelon i+2, which would be *Elementary Part*, *Sub-Assembly* and *Assembly* translated to our specific problem with only the last stage – *Final Assembly* – not shown in this Figure.

It represents all possible solutions to produce a part and potential transportation in between production sites. In this case two suppliers are available per part which could produce the according part. The directed edges between the production sites indicate transportation, but are not visualizing all available transportation links and means in this figure. There are no warehouses between two production sites in this Scenario.

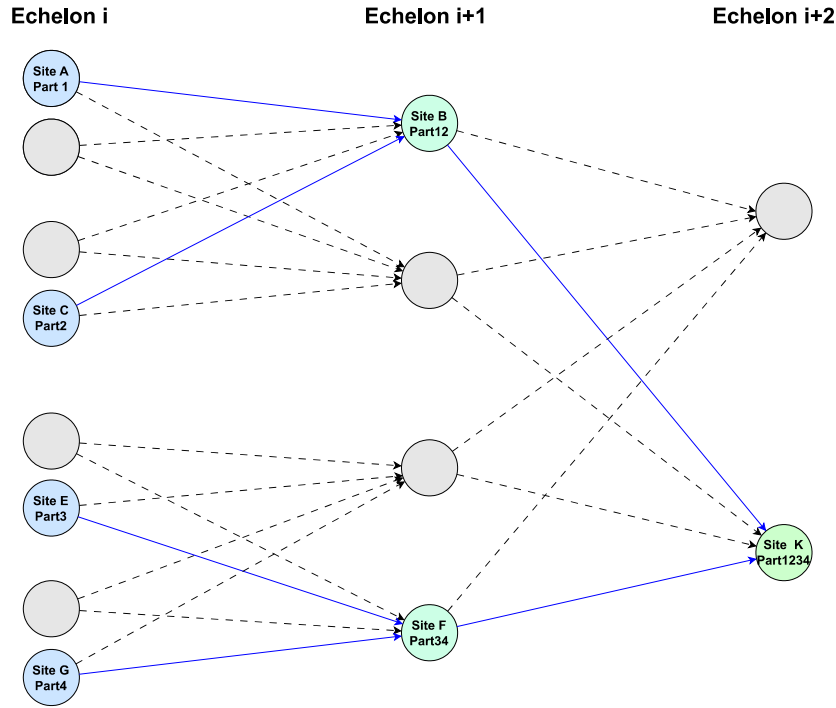


Figure 8: One selected solution with four different paths within the Multi-Echelon Production Network

Figure 8 on the other hand shows one selected solution out of the possibilities seen earlier. The solution includes the choice of one production site per part in case of a *Single Sourcing Strategy* (one production site per part) and the selection of a transportation link and according mean in between two selected production sites of consecutive activities.

A simple single batching is assumed meaning that only one single piece of a specific part is transported by the according transportation mean.

All together, the Double Sourcing Strategy, the Warehouse Strategy for selecting zero to two warehouses and the single batching as defined above sets the baseline configuration for this work.

The predefined KPIs are determining the objective functions which need to be optimized leading to a multi-objective optimization need.

The **Objective KPIs** are *Production Costs*, *Transportation Costs* and *CO₂ emissions*. All are supposed to be conflicting and therefore subject to a Pareto optimization. The objective with the highest priority of this thesis is to find Pareto optimal solutions

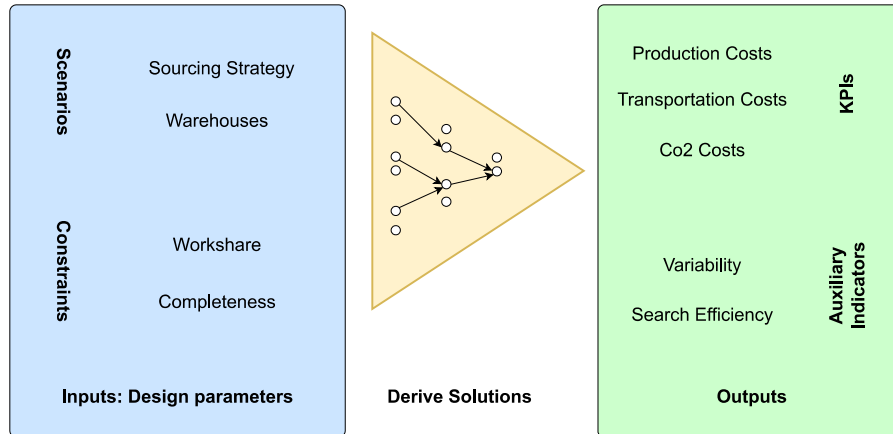


Figure 9: Design parameters and outputs

for some combinations of the new KPIs as stated above.

Moreover, having high variability among answer sets is required for two reasons: first, to increase confidence that a large amount of the search space is explored and no better local optimum is missed, and second, to provide a variety of solutions that an industrial architect can actively explore to learn from unknown solutions and gain additional knowledge which can support an iterative and incremental product development.

Figure 9 gives an overview of the inputs and outputs of the experiments to be performed in this thesis. The inputs are considered design variables of the industrial system and the outputs are KPIs or auxiliary indicators to measure the quality of the solution system.

The optimization of the industrial system as above is a task performed in the context of systems engineering activities. Modeling and simulation capabilities as developed and provided in this thesis aim at providing decision support for design trades [12]. In order to take decisions, it is not only required to obtain results for

the KPIs, but also to have means to interpret the results properly. Therefore a visualization of the solutions and results is a desired, but not mandatory function to be delivered with this thesis.

The research questions derived from the problem statements and objective are:

Research questions

1. Does a weighting of KPIs regarding the objective functions lead to more variability of the answer sets and support finding Pareto optimal solutions?
2. Do heuristic programming and *Weak Constraints* help find solutions in more promising areas as expected by domain knowledge and therefore faster generate improved results with regards to the optimization objectives?
3. Can a combined use of all above-mentioned methods lead to significantly better answer sets with regards to the optimization objectives, with more variability of answer sets and lower runtime?

1.5 Current State of Research

ASP has proven to deliver very good results when it comes to solving search problems in an industrial domain . There are successful industrial implementations mainly in the application areas of configuration, scheduling and planning. The declarative problem specifications are easy to understand and provide high expressiveness, making it a suitable paradigm [6]. Erdem, Gelfond and Leone [13] provide an overview of successful implementations, for instance in the optimization of the planning of Robot activities, where the work plan is considered optimal when the total cost of actions is minimal. Ostrowski, Schaub, Toletti and Wanko have presented an ASP based solution for the problem of train scheduling involving routing and scheduling optimization for a dense railway system.

As with imperative programs, ASP often faces a huge search space with many non-deterministic decisions. The *clingo* solution offers non-domain and domain heuristics to improve the performance. Non-domain heuristics have not demonstrated a significant increase of performance yet, therefore research is focusing more on introducing domain-specific heuristics into ASP solvers [6].

Combinatorial optimization problems are usually solved in a single shot [3] which leads to limitations in the exploration of the search space. Very few multi-shot approaches exist, most of them focus on dealing with changing logic programs. They tackle the grounding and solving steps with a flexible approach that enables the manipulation of logic programs with certain operations [14]. This approach can also help in reducing the ground programs to a minimal size which is very beneficial as the grounding of very large problem instances is one of the major challenges in the use of ASP for industrial decision problems.

The use of multi-shot approaches is not officially published yet for the *Potassco* framework.

To the best of our knowledge there are no references found that investigate the capabilities of a system that combines active optimization statement weighting and domain-heuristics.

1.6 Outline

This subsection provides an overview and the structure of this thesis and how the sections interconnect.

The thesis starts with an *Abstract* which serves as a brief preview. It summarizes the motivation, key problems, methodologies, major findings and conclusions.

The *Introduction* outlines the motivation behind the research and provides necessary background information. It includes the following content:

- *Motivation and Background*: A description of the relevance of the research area and the broader context of this work
- *Problem Statement*: A formulation of the general and specific problems to be addressed in this thesis
- *Related Problems*: A mentioning of similar or related problems
- *Objectives*: The research goals and expected outcomes
- *Current State of Research*: A short overview of previous and related works, the current state and possible gaps in the literature

The *Theoretical Background* will review the theoretical foundations necessary to understand this work. It is divided as follows:

- *Logic Programming*: An introduction to the fundamental principles of logic programming
- *Answer Set Programming and Stable Model Semantics*: An explanation of answer set programming as a paradigm including the stable model semantics as an essential element of it
- *Potassco Toolset*: A description of the software tools and frameworks utilized for this work
- *Input Language*: An overview of the input language and the according constructs used for modeling and representation of the problem in this thesis

The section **Representation of the Industrial System and Optimization Problem** is essential to understand the logic program created in this work, which is one of the core elements of this thesis and translating the real-world problem in a computational one. It consists of the following topics:

- **Issues of the Initial Program:** Explaining some major issues that caused the initial encoding to not produce the expected results or even errors
- **Representation of the Logistics System in a Logic Program:** Presenting the Problem Encoding with facts and rules
- **New Features and Expected Impact:** Introducing the new features to be tested and their expected impact on the objective functions and search tree traversal behavior

The *Methodology* section outlines the scientific approach adopted in this work. It explains:

- the specific research method and experimental design employed
- the evaluation criteria and procedures for analyzing the results

The outcomes of the experiments of the main phase and final phase are presented in the *Results* sections. They are supported by providing figures, tables and statistical data.

Both Results sections are followed by their associated Discussions which will interpret the results in the context of the research questions and objectives. It critically reviews the findings and highlights the limitations of the work.

The thesis will close with the *Conclusion* which will summarize the main findings and reflect the contributions. It will also include a suggestion for *Future Work*.

2 Theoretical Background

From an imperative programming perspective and in a simplified way, the industrial optimization problem described above could be characterized as finding the shortest path with constraints in a DAG where the nodes represent manufacturing sites and the edges transportation among them. The nodes hold the information about Production Costs while the edges provide the Transportation and CO₂ Costs. In this work we look at the problem from a declarative programming perspective using ASP as the programming methodology. It is rooted in the field of artificial intelligence and computational logic [15].

2.1 Logic Programming

Declarative programming characterizes the expected solution of a problem instead of precisely defining the steps to get there [15]. One kind of declarative programming is *Logic Programming* which is the foundation of the ASP solution used in this thesis. In Logic Programming an algorithm can be regarded as consisting of two components, the logic and the control. The logic expresses what the problem is while the control specifies the control flow. This separation does bring the advantage that the programmer can focus on specifying the logic by describing the problem. The control flow can be left to the Logic Programming system [16]. The implementation of the logic is based on *First-Order Logic* and uses logical formulas to represent knowledge in the form of rules. A Logic Program is a finite set of *rules*. Given a signature Σ , a *rule* r has the following form:

$$A \leftarrow A_1, \dots, A_m, \text{not} A_{m+1}, \dots, \text{not} A_n \quad (1)$$

Let $\text{head}(r) = A$ be the *head* and $\text{body}(r) = A_0, \dots, A_m, \text{not} A_{m+1}, \dots, \text{not} A_n$ be the *body* of the *rule*. Negative atoms are denoted *not* which refers to *Negation as a Failure*, which will be explained later. Therefore the understanding of the formula is that the head of the rule must hold if any positive atom in the body is provably *true* and any negative atom can not be proven *true* and therefore is assumed *false* [17]. Van Gelder [18] defines the *body* of such *rule* as a *sub-goal*. without a body or sub-goal are called *facts* and the ' $\leftarrow$ ' can be omitted.

We define the set of positive atoms in the body of the rule above as

$$B(r)^+ = \{A_0, \dots, A_m\}$$

and the set of negative atoms as

$$B(r)^- = \{\text{not } A_{m+1}, \dots, \text{not } A_n\}$$

The handling of negation can be treated differently and it will be subject to Section 2.2.

Alternatively a rule as in Equation 1 can be written without negation in literal form. A literal can be a positive or negative atom. The rule looks as following accordingly:

$$A \leftarrow L_1, \dots, L_n \quad (2)$$

When comparing Equation 2 with the previous style in Equation 1, L_0 corresponds with the head atom A , the literals $L_1, \dots, L_m$ with the positive atoms $A_1, \dots, A_m$ and $L_{m+1}, \dots, L_n$ with the negative atoms $A_{m+1}, \dots, A_n$.

When a rule does not contain any variables, but only constants of the signature Σ , it is called a *ground rule*. The *ground program* of a program that includes variables can be obtained by replacing all variables in the program by all combinations of constants provided with the signature of the program. The *universe* of such kind of program is consequently built by the *ground terms* of the given signature, the *Herbrand-Universe*.

Imagine the following signature $\Sigma = \{Hamburg/0, Toulouse/0, Part1/0, Part2/0\}$. All elements are functions of arity 0, i.e. constants. The first two represent production locations while the latter two represent parts to be manufactured. Then the according Herbrand-Universe is given with $U = \{Hamburg, Toulouse, Part1, Part2\}$. For the following program

Example 1.

```

ispart(Part1)
isPart(Part2).
isProduction(Hamburg).
isProduction(Toulouse).
canProduce(Hamburg, Part1).
canProduce(Toulouse, Part2).
produces(X, Y) ← canProduce(X, Y), isProduction(X), isPart(Y).

```

the according ground rules are

```

isPart(Part1).
isPart(Part2).
isProduction(Hamburg).
isProduction(Toulouse).
canProduce(Hamburg, Part1).
canProduce(Hamburg, Part2).
canProduce(Toulouse, Part1).
canProduce(Toulouse, Part2).
produces(Hamburg, Part1) ← canProduce(Hamburg, Part1),
                             isProduction(Hamburg), isPart(Part1).
produces(Hamburg, Part2) ← canProduce(Hamburg, Part2),
                             isProduction(Hamburg), isPart(Part2).
produces(Toulouse, Part1) ← canProduce(Toulouse, Part1),
                             isProduction(Toulouse), isPart(Part1).
produces(Toulouse, Part2) ← canProduce(Toulouse, Part2),
                             isProduction(Toulouse), isPart(Part2).

```

See in Example 1 that the variables in the last rule of the program in Example 1 are replaced by some - but not all - combination of the elements of the Herbrand-Universe. Note that the process of grounding already mentions the possible domains of the variables which is defined by the facts. It is obvious that only the constants which are defined as production sites ($isProduction(Hamburg), isProduction(Toulouse)$) can produce parts ($isPart(Part1), isPart(Part2)$). The consideration of the domain of a variable while grounding can reduce the effort to generate the ground programs. This aspect of the definition of the domain of a variable is especially critical if the body of a rule contains negation. This issue will be addressed later with the concept of *safe* and *unsafe negation*.

Herbrand-Interpretations I are assignments of truth values $\{\perp, \top\}$ to a set X of ground atoms indicating valid solutions, also called *Herbrand-Models*, so that for each rule in P $I(r) = \top$ holds [14, 1]. For the definition of a valid solution or model, there are varying concepts depending on the selected semantics. In general, different interpretations I may be models of program P , but some solutions might be more favorable. One common problem that requires attention is the handling of incomplete information. These topics will be addressed in the Section 2.2 for further details.

A program consisting only of positive atoms and ground formulas is called a *Pos-*

itive Propositional Program [15].

Positive Propositional Programs are *stratified*, which ensures that an atom never depends on its own negation. *Non-stratified* programs on the other hand lead to cyclic dependencies putting additional complexity to the programs, valid interpretations and the possible models [18].

Furthermore the concept of safe and unsafe negation in the body or sub-goal of a rule is important to avoid an undefined domain of a variable. Unsafe negation occurs when a negative sub-goal contains free variables. In order to avoid unsafe negation it is necessary that any variable in a negative sub-goal must appear in the head of the rule or in a positive sub-goal. Otherwise the undefined domain will lead to a free variable for the negative atom and can increase the search space significantly for grounding and proof of negation, practically reducing the performance of the solver [18].

Example 2.

$$\begin{aligned} \text{unproduciblePartAt}(X, Y) &\leftarrow \text{part}(X), \text{not partProducibleAt}(X, Y). \\ \text{unproduciblePartAt}(X, Y) &\leftarrow \text{part}(X), \text{productionSite}(Y), \\ &\quad \text{not partProducibleAt}(X, Y). \end{aligned}$$

Example 2 gives two examples of formula - one safe and one unsafe. The first formula contains the free variable Y in a negative atom of its body without defining it with any positive atom while the second formula defines the variable Y with the preceding positive atom $\text{productionSite}(Y)$ and is therefore safe.

2.2 Answer Set Programming (ASP) and Stable Model Semantics

ASP is a special paradigm of logic programming without a specific query that needs to be answered. Instead, ASP returns an *Answer Set* that includes all facts that can be derived from the rules of the program. This answer set is the *Stable Model* of the program [15]. The Stable Model semantics is one main characteristic of ASP. Stable Model semantics handles incomplete information under the *Closed World Assumption* (CWA) applying *Negation as a Failure*. This means that any ground atom that can not be derived from the program P is assumed false by default. When applied to a positive propositional program, the Stable Model is a Minimal Model, meaning that there exists no subset of the answer set which is also a Stable Model of the program [15].

Positive propositional programs have a unique *Minimal Herbrand Model* [19].

Most ASP solvers use a two-step approach as shown in Figure 10. First, a ground program of the existing program is created. This step is called *grounding*. Then the ASP solver computes the Stable Models of this propositional program - the Answers Sets [14, 3].

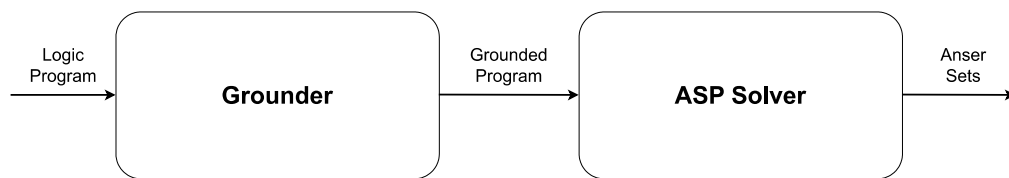


Figure 10: Typical two-step architecture of ASP solutions as *clingo*, Source: [3]

The grounding process requires the substitution of variables by constants which can lead to a combinatorial challenge with a complexity depending on the specific problem encoding. This can lead to the so-called *Grounding Bottleneck* [3]. Drivers of complexity can be the number of variables and elements of the universe, but also the use of certain input language constructs that reflect the combinatorial challenges of the problem. Possible constructs of the used input language leading to possibly increased complexity will be presented later.

To tackle the possible Grounding Bottleneck, ASP systems might apply strategies to delay grounding of some parts of the program by incremental grounding of program segments or even preventing from grounding parts that are not needed at all for solving. The latter is presented by Faber and Friedrich in [3] and named *Lazy Grounding*.

For solving, SAT solving techniques are applicable on propositional programs, looking for an assignment of the variables making the entire formula true. These SAT solving techniques are often applied as well for ASP solutions [17].

Common algorithms to solve the Boolean Satisfiability Problem are DPLL (*Davies-Puttnam-Logemann-Loveland*) and CDCL (*Conflict-Driven Clause Learning*). Both algorithms use *Backtracking*, *Unit Propagation* and *Restarts* as key mechanisms to find solutions. Backtracking is a method in an algorithm to return to an earlier decision level when the recent search was unsuccessful. It is a mean to traverse the search tree. Unit propagation is a deterministic rule which looks for unit clauses - clauses with a single unassigned literal inside the logic program which can be assigned correctly with little effort. Restarts is a technique that can help improve the efficiency of a search by resetting the assignment of variables, but keeping learned clauses or *Nogoods*, which will be explained later. As a result the search tree will be traversed differently when starting from the beginning [20]. The foundations and further explanations of DPPL and the according use of Backtracking and Unit Propagation can be found in [21] and [22]. The concept of *conflicts*, i.e. assignments that can not lead to solutions, is important to both algorithms. The main difference between both algorithms is that for CDCL, after finding a conflict, not only backtracking is performed, but adding a new learned clause to the program which represents the assignment which led to the conflict and therefore can not lead to a valid solution. Also, there is a difference in the application of backtracking. DPLL performs chronological backtracking while CDCL uses non-chronological backtracking by recording the causes of conflict and using it later instead of immediately returning to the preceding decision level. This can prune the search space significantly [23, 24]. ASP is able to solve decision problems that are NP-hard and harder [3].

2.3 Potassco Toolset

The toolset used in this work is the *Potassco Collection* which includes *clingo*, a combination of a solver, which is called *clasp*, and a grounder named *gringo*. As a conse-

quence, *clingo* follows a two-step approach to generate answer sets as described earlier. First, a grounding step is performed replacing all variables in the logic program with given instances and generating a finite propositional representation of the program [25]. Then the solver computes the stable models using *CDNL* (*Conflict-Driven Nogood Learning*) techniques which can be regarded as an evolution of the CDCL algorithm described in the previous section [14].

2.4 Input Language of *clingo*

A logic program in *clingo* allows the use of facts and rules as described in Section 2.1. Additionally, so called **Integrity Constraints** are admitted. Integrity Constraints are rules without a head of the form:

$$: - A_1, \dots, A_m, \text{not} A_{m+1}, \dots, \text{not} A_n \quad (3)$$

An Integrity Constraint as in Equation 3 states that solution candidates which satisfy the body are not desired solutions, potentially describing constraints to the solution space.

clingo incorporates the utilization of **Weak Constraints** to provide guidance throughout the search process. Weak Constraints diverge from the binary nature of Integrity Constraints as they are not restricted to absolute fulfillment or negation. Instead, they operate with optimization statements that enable the evaluation of constraint adherence with regard to specific criteria. By this, the task of finding valid solutions - or answer sets - can be extended to finding optimal solutions [26].

Weak Constraints have the form:

$$:\sim L_1, \dots, L_n [w@p, t_1, \dots, t_n] \quad (4)$$

The Weak Constraint is associated with a term tuple which can be seen in the squared brackets of Equation 4. The first element of the tuple represents the weight that is associated with the satisfied body. The $@p$ is an optional prioritization factor. Whenever the body of a Weak Constraint is satisfied, it contributes its term tuple $t_1, \dots, t_n$ to a cost function. This cost function accumulates the weights w of all contributing tuples. Each tuple is only included once with its dedicated weight which refers to the **aggregate** property of the Weak Constraint. Note that when generating term tuples which contribute to some function, this is normally done to perform some operations on the results. This is demonstrated later when explaining optimization statements.

To properly understand the underlying concepts behind Weak Constraints it is important to see how aggregates are defined and expressed in *clingo*.

Before looking at some examples of aggregates and arithmetic functions applied on them, it is important to mention that typically the set of tuples in aggregates as

described above are subject to conditions. Therefore **conditions** and **conditional literals** will be presented upfront.

A conditional literal has the form:

$$L_0 : L_1, \dots, L_n \quad (5)$$

The ':' expresses a set notation and the literals behind it - $L_1, \dots, L_n$ - is the condition. L_0 holds whenever the condition holds. Conditions can have a meaning of a nested implication if we look at the next example:

$$A : - L_0 : L_1, L_2 \quad (6)$$

In the body of Equation 6, L_0 and L_1, L_2 practically act as head and body. L_0 holds if L_1 and L_2 hold.

Aggregates are expressive modeling constructs that allow forming values from groups of elements and conditions over terms. Together with arithmetic functions and comparisons available from build-in light theory solving capabilities, truth values can be derived from the aggregate's evaluation [26]. Aggregates can appear in the head and body of rules.

An aggregate in the body of a rule has the form:

$$s_1 \prec \alpha\{t_1 : L_1; \dots; t_n : L_n\} \prec s_2 \quad (7)$$

In Equation 7 the main part of the aggregate is inside the brackets including the term tuples $t_1, \dots, t_n$ and the corresponding conditional literal tuples $L_1, \dots, L_n$. α is some predefined built-in arithmetic function that can be applied to the term-tuples. *clingo* provides the functions *#count* (counts the number of term tuples), *#sum* (sums the weights of the term tuples), *#sum+* (sums only the positive weighted elements of the term tuples), *#min* (the minimum value weight of the term tuples) and *#max* (the maximum value of the term tuples).

The result of applying the function α to the term tuples can be compared to the terms s_1 and s_2 which act as lower and upper bounds. The comparison finally can be evaluated to *true* or *false* values.

The bounds as well as the function α can be omitted. In that case the default is the *#count* function, which is resulting in a so called *Cardinality Constraint*. There are no weights needed for the elements of the term tuples of a Cardinality Constraint. For the *#count* function a default value of 0 is set as a term in case no term is provided. It is important to remind that the term tuples are treated as set elements, so that the same term tuple is only counted once. Taking into account the said, the shortcut of the aggregate above would be:

$$\{t_1 : L_1; \dots; t_n : L_n\}2 \quad (8)$$

The aggregate in Equation 8 can be understood as selecting maximum two elements t_i and t_j which fulfill the corresponding literal condition L_i .

Example 3.

$$1\{partProducedAt(Part, Location) : partProducibleAt(Part, Location)\}2$$

Example 3 shows an example of a Cardinality Constraint with the *#count* function. The term tuple as well as the function declaration is therefore omitted. This aggregate will select minimum one or maximum two production locations for every part (*partProducedAt(Part, Location)*) which fulfills the condition that the according location is capable of producing it (*partProducibleAt(Part, Location)*).

Cardinality Constraints are also called *Choice Rules* as they enable to select a predefined number of elements out of a set. In case that no boundaries are set, all possible combination of the set can be selected.

When it comes to aggregates in the head of a rule, the literal to be derived needs to be added. Therefore a head aggregate has the form:

$$s_1 \prec \alpha\{t_1 : L_1 : L_1; \dots; t_n : L_n : L_n\} \prec s_2$$

Optimization statements are a way to express several Weak Constraints. There are two optimization statements available, *#minimize* and *#maximize*.

A minimize statements for example has the form:

$$\#minimize\{w_1@p_1, t_1 : L_1, \dots, w_n@p_n, t_n : L_n\}$$

It represents the following n Weak Constraints:

$$\begin{aligned} &:\sim L_1.[w_1@p_1, t_1], \dots, \\ &:\sim L_n.[w_n@p_n, t_n] \end{aligned}$$

A *#maximize* statement is treated the same way as a *#minimize* statement with inverse weights.

With the use of an optimization statement with several Weak Constraints, a weighting or order of prioritization of term tuples can be performed. The definition of an *optimal answer set* can be adjusted to a specific preference this way.

The following example should showcase the use of a *#minimize* optimization statement applied on a *#sum* aggregate.

Imagine there is a transportation path between two production locations expressed this way as a fact: *path(Part, Location1, Location2, Distance)*. It tells that the Part can be transported from Location 1 to Location 2 with the according distance. If we aim to minimize the transportation distance of all necessary transportation between two consecutive production steps performed at the according locations, it can be expressed with the following statement:

Example 4.

$$\#minimize\{(Distance, Part, Location1, Location2 : \\ path(Part, Location1, Location2, Distance)\}.$$

This statement will sum all the single distances reflected in the term *Distance* of ground atoms that hold the condition $path(Part, Location1, Location2, Distance)$ and find the model that minimizes this value. Note, that the value to be minimized is the first element of the term tuple - in this case the term *Distance* from the term tuple $Distance, Part, Location1, Location2$.

The other elements of the term tuple are needed as some kind of identifier to address the aggregate property of the expression and to prevent from counting elements only once when it is not intended. The following example will demonstrate the potential issue:

Example 5.

$$\#minimize\{(Distance, Location1, Location2 : \\ path(Part, Location1, Location2, Distance)\}.$$

Imagine, the term *Part* is left out from the term tuple compared to Example 4. Then the two different ground atoms $path(Part1, Hamburg, Toulouse, 2000)$ and $path(Part5, Hamburg, Toulouse, 2000)$ do both contribute with the identical term tuple $2000, Hamburg, Toulouse$ to the $\#sum$ function and therefore are only counted once due to the aggregate property of the construct. Assuming that these two are the only term tuples contributing to the $\#sum$ function, it evaluates to 2000 while in the previous example it evaluates to 4000. In this specific case, the additional term *Part* ensures that despite the fact that it is a transport between the same production locations it distinguishes them, because both represent two different transportation activities for different parts at possibly different stages of the production process.

2.4.1 The Grounder *gringo*

The Grounder used in the *Potassco* framework is called *gringo*. It is applying incremental grounding by gradually increasing the atom base. Facts and rules that can be derived independently can be grounded immediately while others are on hold until their conditions are met. This can have a positive impact on grounding efficiency and runtime overall.

2.4.2 The Solver *clasp*

The SAT solver used in the *Potassco* framework is *clasp*. The solving mechanisms in *clasp* are based on algorithms such as DPLL and CDCL including Unit Propagation, Backtracking and conflict-driven learning techniques which guide the search tree traversal and prune the search space. *clasp* is an evolution of the previously

mentioned algorithms believing that all inferences in ASP solving can be regarded as Unit Propagation of so-called *Nogoods* [17]. Nogoods are local assignments that can not be extended to a solution. Formally, a Nogood is a set of literals $\{\delta_1, \dots, \delta_n\}$ defining an assignment that is unintended as it can not lead to a solution. A Nogood is *violated* by an assignment A if $\{\delta_1, \dots, \delta_n\} \subseteq A$. Otherwise, if $\{\delta_1, \dots, \delta_n\} \not\subseteq A$, A is a solution for a set of Nogoods [17].

When it comes to heuristic decisions to be made, clasp uses the *VSIDS* (*Variable State Independent Decaying Sum*) heuristic as its default [26]. VSIDS is a heuristic that selects the ground literals depending on their values inside the (local) program. The value is resulting from the literal's appearance in recent conflicts. Furthermore it uses a decaying sum mechanism to adjust the scores or values dynamically over time. For further details consult [27].

One important feature of *clasp* is the possibility to use **Domain-specific Heuristics** into ASP solving which complements the solver's default heuristics. This enables to guide the search through non-determinism with experts knowledge and explore promising zones of the search space first, potentially leading to a more efficient search [26].

Heuristics are represented the following way:

$$\#heuristic A_0 : L_1, \dots, L_n [w@p, m] \quad (9)$$

The $\#$ symbol is used to signal meta-statements in *clingo*. A_0 is an atom, $L_0, \dots, L_n$ is the body of a rule, and w, p and m inside the squared brackets are terms. Similar as for Weak Constraints, w represents a weight, p is an optional prioritization and m is a modifier representing different types of heuristic information. Six different types for modifier m are available in *clingo*: *sign*, *level*, *true*, *init* and *factor*.

The heuristic modifier *sign* gives guidance for the assignment of the truth value when the solver has to decide. By default the sign value is 0. If the sign value - represented by w above - is set greater than 0, then the solver will assign the value of the according atom to *true*. If the value is less than 0, it will be set to *false*. These decisions will not impact the number of answer sets, but only the order in which they will be identified and listed. To understand this concept, the modifier *level* will be introduced as well and an example will demonstrate the impact on according answer sets. The heuristic modifier *level* puts some kind of importance to the atoms. In the following example, the solver is asked to assign two atoms with the truth value *true* first. The integrity constraint disallows to have both true at the same time. So, the modifier *level* determines which atom to set *true* first, i.e. *transportMean(truck)* in this case:

$$\begin{aligned}
& \#heuristic\ transportMean(truck). [1, sign] \\
& \#heuristic\ transportMean(aircraft). [1, sign] \\
& \#heuristic\ transportMean(truck). [10, level] \\
& \{transportMean(truck); transportMean(aircraft)\} \\
& : - transportMean(truck), transportMean(aircraft).
\end{aligned} \tag{10}$$

Equation 10 will result in providing the answer set $\{transportMean(truck)\}$ first as it is asked to be set to *true* due to its *sign* value of 1 and its higher *level* value than $transportMean(aircraft)$. The next answer set is $\{transportMean(aircraft)\}$ and the final answer set is the empty set. The answer set with both atoms is disallowed due to the integrity constraint.

The modifier *true* combines the modifiers *sign* and *level*. The detailed explanations of the other modifiers will be left out here as they will not be used for this thesis. The interested reader is referred to [26].

clingo offers - among others - a vast set of additional features, including advanced constraint and theory solving capabilities, optimization and hybrid solving techniques. Some of these capabilities are still in the research stage [26].

3 Representation of the Logistics System and the Optimization Problem

This section will present how an industrial and logistics system – including the optimization problem and constraints – can be represented in a logic program with its according facts and rules. This program is considered to be the *baseline* program. Significant issues that are resolved with the new baseline and the according changes of the encoding will be highlighted next.

This chapter will end with the features to be tested and their according formulations in ASP.

3.1 Representation of the Logistics System

As common in logic programming and ASP, the program can be split into facts and rules and structured in a *Guess-and-Check* architecture [28]. Guess-and-Check is a structure which aims at finding solutions by first generating solution candidates (*Guess*) and then filtering these by eliminating those that do not meet constraints or conditions (*Check*).

The baseline program including facts and rules is defined as the baseline configuration labeled with **(baseline)**.

Find an overview of the facts, i.e. 15 predicates, their descriptions and the according labels for further referencing in Table 1. Overall the logic program consists of more than 30.000 instances of facts.

The predicates *productionLocation/1* and *warehouseLocation/1* simply define the domain of existing production and warehouse locations.

costLocation/2 provides the relative cost of working units per location and is required to calculate the Production Costs per part at a specific production location.

country/1 defines the domain of countries where the production and warehouse locations are belonging, while *locatedIn/2* establishes the relation between a location and the country.

part/1 defines the domain of parts to be produced in a dedicated production step and *valueAdded/2* the according value or required workload to be manufactured independent from the production location. *partProduceableAt/2* determines which part can be produced at which production site.

productionPlan/2 provides the production sequence, telling which part is required for another part as an input.

transportMean/1 sets the domain of available transportation means.

canBeTransportedFromTo/8 is the most complex fact providing all available transportation links including the according properties of the link. There are more than 29.0000 facts of this predicate.

The rules to describe a logistics system are explained next.

Fact	Description	Label
<i>productionLocation(aProduction)</i>	aProduction is a Location that can produce a part	(prodLoc)
<i>warehouseLoc(aWarehouse)</i>	aWarehouse is a Warehouse that can store and repack parts	(wareLoc)
<i>costLocation(aProduction, 120)</i>	aProduction has a cost factor of 120 per unit	(costLoc)
<i>country(germany)</i>	Germany is a country	(country)
<i>locatedIn(aProduction, italy)</i>	aProduction is located in Italy	(locIn)
<i>part(part1)</i>	part1 is a part to be produced	(part)
<i>valueAdded(part1, 30)</i>	part1 has a value or workload of 30 units	(valAdd)
<i>partProduceableAt(part1, aProduction)</i>	part1 can be produced at production location aProduction	(pProdceableAt)
<i>productionPlan(part12, part1)</i>	part1 is needed for production of part 12 and needs to be produced before part12	(prodPlan)
<i>transportMean(aircraft)</i>	an aircraft is a transportation mean	(transpMean)
<i>canBeTransportedFromTo(Hamburg, Toulouse, part2, truck, 120, 70, 23, 1200)</i>	part2 can be transported from Hamburg to Toulouse by truck with distance 120, lead-time 70, Co2 emission 23 and transport cost 1200	(canBeTransp)
<i>elementaryPart(part1)</i>	part1 is an elementary part and therefore belonging to the first stage of the multi echelon network	(isElementary)
<i>subAssembly(part12)</i>	part12 is a sub-assembly part and therefore belonging to the second stage of the multi echelon network	(isSubAss)
<i>assembly(part1234)</i>	part1234 is an assembly part and therefore belonging to the third stage of the multi echelon network	(isAss)
<i>finalProduct(part12345678)</i>	part12345678 is an elementary part and therefore belonging to the last stage of the multi echelon network	(isFinal)

Table 1: Overview of all facts of the logic program.

Find the overview of the baseline program rules – referred to as the baseline configuration with the label (**baseline**) in Table 2. The following explanations are providing the according label of the rules which are shown in the *label* column in the table.

At every production location there is an *intrasite* transportation available that can transport any part within a location with distance zero. As a consequence, every part can be transported intrasite: (**intraSite**).

It is always possible to transport a part directly from one production location to another if it can be transported at all: (**ruleDirect**).

Another option is that the part goes via a Hub, which is a warehouse: (**ruleVia1**). Or the part can even be transported via two hubs, which significantly increases accessibility: (**ruleVia2**).

Finally, the solution candidates, i.e. the valid logistics system to be derived, are represented by two predicates, *path/8* and *partProducedAt/2*. Therefore these rules reflect the *guess* part of the baseline program as described earlier.

partProducedAt/2 describes which part is produced at which site. Depending on the Sourcing Strategy to be selected for the according echelon, it is expressed as a Choice Rule/Cardinality Constraint. It is the implementation of the logical element which represents the *Set Cover Problem* as described in Section 1.2.1.

It must be possible to define the Sourcing Strategy per production stage considering the multi-echelon property of the production and logistic system. Therefore,

the Choice Rules are implemented for every level of the **multi-echelon production network**: (**choiceLv1**), (**choiceLv2**), (**choiceLv3**) and (**choiceLv4**). This ensures that every part is produced at at least at the number of sites that was set as the lower bound. The baseline value is 2 for all lower and upper bounds resulting in a double sourcing request for all production stages.

One important constraint is the implementation of the **workshare** constraint ensuring that every production location must at least produce one part. In the (*baseline*) configuration, this is implemented as an integrity constraint: (**workshare-IC**).

The second solution candidate predicate, *path/8*, expresses a transportation path between two production sites that needs to be performed with a specific transportation mean and the according attributes. As this is part of the solution to be derived, it is expressed as a Choice Rule/Cardinality Constraint in the head of the rule. A valid path can be either direct, via one hub or via two hubs. One out of several possible paths taking into account the correct production sequence, including the choice of an available transportation mean, has to be selected: (**choicePath**).

Finally, the optimization statements used for the objective functions - or KPIs - can be found in the group of rules (**optim-baseline**).

The first optimization statement aims at minimizing the Production Costs, the second the Transportation Costs and the last one the CO₂ emissions. As no priorities are set, *clingo* does prioritize in declining order of appearance. This topic will be detailed later in section 3.2.1.

3.2 Issues of the initial Research Work

This thesis builds upon the initial research work done by the central R&T team and it is worth to specifically look at some major shortcomings or issues in order to understand the rationales behind some modified or new features implemented for the baseline or to be tested in the experiments.

3.2.1 Optimization Statements

The initial problem encoding which is part of the baseline configuration is facing general issues with the formulation of the optimization problem, making it unsuitable for a Pareto optimization and resulting in wrong results for some KPIs. This needed to be resolved together with logical mistakes encountered.

The attempt for a Pareto optimization for the KPIs as mentioned above was realized by three independent *#minimize* statements as visible in (**optim-baseline**).

clingo will prioritize by the order of appearance by default, meaning that the first optimization statement will be optimized as first priority. Once this term is optimized, the following optimization statements will be taken into account to break the tie of the previous optimization [26]. As a conclusion, this might lead to a Pareto optimization with a behavior that prioritizes accordingly and potentially generates answer sets with unbalanced KPIs, meaning that the KPI with the highest priority

Program excluding facts	Description	Label
$\text{transportMeanAtSite}(P, \text{intrasiteTransport}) \leftarrow \text{productionLocation}(P)$ $\text{canTransport}(\text{intrasiteTransport}, \text{Part}) \leftarrow \text{part}(\text{Part})$	intrasite transport available at every site intrasite can transport every part	(intraSite)
$\text{direct}(\text{Part}, \text{From}, \text{To}, \text{TM}, \text{D}, \text{LT}, \text{CO2}, \text{TC}) \leftarrow \text{canTransport}(\text{From}, \text{To}, \text{Part}, \text{TM}, \text{D}, \text{LT}, \text{CO2}, \text{TC})$ $\text{via1}(\text{Part}, \text{From}, (\text{Via1}, \text{To}), (\text{TM1}, \text{TM2}), \text{D}, \text{LT}, \text{CO2}, \text{TC})$ $\leftarrow \text{canTransport}(\text{From}, \text{Via1}, \text{Part}, \text{TM1}, \text{D1}, \text{LT1}, \text{CO21}, \text{TC1}), \text{From!} = \text{To}, \text{productionLoc}(\text{From}),$ $\text{productionLoc}(\text{To}), \text{warehouseLoc}(\text{Via1}), \text{canTransport}(\text{Via1}, \text{To}, \text{Part}, \text{TM2}, \text{D2}, \text{LT2}, \text{CO22}, \text{TC2}),$ $\text{D} = \text{D1} + \text{D2}, \text{LT} = \text{LT1} + \text{LT2}, \text{CO2} = \text{CO21} + \text{CO22}, \text{TC} = \text{TC1} + \text{TC2}$ $\text{via2}(\text{Part}, \text{From}, ((\text{Via1}, \text{Via2}), \text{To}), (\text{TM1}, \text{TM2}, \text{TM3}), \text{D}, \text{LT}, \text{CO2}, \text{TC})$ $\leftarrow \text{canTransport}(\text{From}, \text{Via1}, \text{Part}, \text{TM1}, \text{D1}), \text{From!} = \text{To}, \text{productionLoc}(\text{From}), \text{warehouseLoc}(\text{Via1}),$ $\text{canTransport}(\text{Via1}, \text{Via2}, \text{Part}, \text{TM2}, \text{D2}), \text{Via1!} = \text{Via2}, \text{productionLoc}(\text{To}), \text{warehouseLoc}(\text{Via2}),$ $\text{canTransport}(\text{Via2}, \text{To}, \text{Part}, \text{TM3}, \text{D3}), \text{D} = \text{D1} + \text{D2} + \text{D3}, \text{LT} = \text{LT1} + \text{LT2} + \text{LT3},$ $\text{CO2} = \text{CO21} + \text{CO22} + \text{CO23}, \text{TC} = \text{TC1} + \text{TC2} + \text{TC3}$	transport part directly via one warehouse or via two warehouses	(ruleDirect) (ruleVia1) (ruleVia2)
$\leftarrow \text{productionLoc}(P), \text{not produced}(P)$ $\text{produced}(P) \leftarrow \text{producedAt}(_, P)$	at least one part per site true if P produces a part	(workshare-IC)
$1 \{ \text{path}(\text{Part}, \text{From}, \text{To}, \text{TM}, \text{D}, \text{LT}, \text{CO2}, \text{TC}) : \text{direct}(\text{From}, \text{To}, \text{Part}, \text{TM}, \text{D}, \text{LT}, \text{CO2}, \text{TC});$ $\text{path}(\text{From}, \text{Part}, (\text{Via1}, \text{To}), (\text{TM1}, \text{TM2}), \text{D}, \text{LT}, \text{CO2}, \text{TC}) :$ $\text{via1}(\text{Part}, \text{From}, (\text{Via1}, \text{To}), (\text{TM1}, \text{TM2}), \text{D}, \text{LT}, \text{CO2}, \text{TC}),$ $\text{path}(\text{Part}, \text{From}, ((\text{Via1}, \text{Via2}), \text{To}), (\text{TM1}, \text{TM2}, \text{TM3}), \text{D}, \text{LT}, \text{CO2}, \text{TC}),) :$ $\text{via2}(\text{Part}, \text{From}, ((\text{Via1}, \text{Via2}), \text{To}), (\text{TM1}, \text{TM2}, \text{TM3}), \text{D}, \text{LT}, \text{CO2}, \text{TC}),) \}$ 1 $\leftarrow \text{producedAt}(\text{Part}, \text{From}), \text{productionPlan}(\text{Super}, \text{Part}), \text{producedAt}(\text{Super}, \text{To})$	direct path, or path via one site, or path via two sites	(choicePath)
$e \{ \text{producedAt}(\text{Part}, P) : \text{produceableAt}(\text{Part}, P) \} e \leftarrow \text{elementary}(\text{Part})$ $s \{ \text{producedAt}(\text{Part}, P) : \text{produceableAt}(\text{Part}, P) \} s \leftarrow \text{subassembly}(\text{Part})$ $a \{ \text{producedAt}(\text{Part}, P) : \text{produceableAt}(\text{Part}, P) \} a \leftarrow \text{assembly}(\text{Part})$ $f \{ \text{producedAt}(\text{Part}, P) : \text{produceableAt}(\text{Part}, P) \} f \leftarrow \text{finalProduct}(\text{Part})$	depending on its echelon each part needs to be produced at e, s, a or f sites, with $e, s, a, f \in \mathbb{N}$	(choiceLv1) (choiceLv2) (choiceLv3) (choiceLv4)
$\# \text{minimize} \{ (\text{Value} \cdot \text{Cost}) / e_1 \cdot e_2, \text{Part}, P : \text{producedAt}(\text{Part}, P), \text{valAdded}(\text{Part}, \text{Value}), \text{costLoc}(P, \text{Cost}) \}$ $\# \text{minimize} \{ \text{TC}, \text{Part}, \text{From}, \text{To}, \text{TM} : \text{path}(\text{Part}, \text{From}, \text{To}, \text{TM}, \text{D}, \text{LT}, \text{CO2}, \text{TC}) \}$ $\# \text{minimize} \{ \text{CO2}, \text{Part}, \text{From}, \text{To}, \text{TM} : \text{path}(\text{Part}, \text{From}, \text{To}, \text{TM}, \text{D}, \text{LT}, \text{CO2}, \text{TC}) \}$	minimize Production Costss minimize Transportation Costs minimize CO ₂ emissions	(optim-baseline)

Table 2: Relevant rules and optimization statements of the (baseline) program excluding facts.

will have good values while the lower priority KPIs might be weak. 1.2.3.

3.2.2 Usage of Aggregates

A wrong usage of aggregates in the optimization statements as described in Example 4 for Transportation and CO₂ Costs was detected resulting in values that are too high for Multiple Sourcing Strategies. The starting point of the issue is the body B of rule **(choicePath)** in Table 2.

While for a Single Sourcing Strategy there is only one production location selected per part and therefore $partProducedAt(Part, From)$ would only evaluate to *True* for one combination of part and production location, it will be *True* for multiple production locations in the case of a Multiple Sourcing Strategy. The same applies to $partProducedAt(Part, To)$. This results in several combinations per part evaluating to *True* in the body $B(r)$ of this rule and ultimately in selecting multiple valid paths as requested by the head of the rule $H((\mathbf{choicePath}))$.

All these valid paths are taken into account for the $\#minimize$ statements, which are aggregates and therefore sum all elements and their contributing terms as if all parts are taking every single path (see section 2.4). In fact, the overall number of a specific part would be distributed among all existing possible paths. The number of possible paths n_{paths} can be determined by the number of production sites for a dedicated predecessor part $nbSites_{pred}$ and the number of sites for the successor part $nbSites_{succ}$ with:

$$n_{paths} = nbSites_{pred} \cdot nbSites_{succ}$$

See the following example for two parts A and B with A needed for production of B: In the case of single sourcing, the body of the rule **(choicePath)** only evaluates to *True* for

$partProducedAt(Part1, SiteA), productionPlan(Part2, Part1), partProducedAt(Part2, SiteC)$

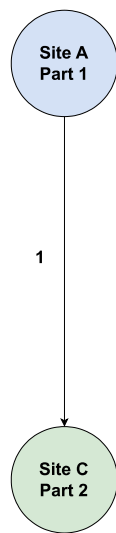
while for the example of double sourcing for $part1$ and triple sourcing for $part2$ all of the following bodies would evaluate to *True* and thus derive the Choice Rule **(choicePath)** to select solution candidates with $path/8$.

Example 6.

$\leftarrow partProducedAt(Part1, SiteA), productionPlan(Part2, Part1), partProducedAt(Part2, SiteC).$
 $\leftarrow partProducedAt(Part1, SiteA), productionPlan(Part2, Part1), partProducedAt(Part2, SiteD)$
 $\leftarrow partProducedAt(Part1, SiteA), productionPlan(Part2, Part1), partProducedAt(Part2, SiteE).$ (11)
 $\leftarrow partProducedAt(Part1, SiteB), productionPlan(Part2, Part1), partProducedAt(Part2, SiteC).$
 $\leftarrow partProducedAt(Part1, SiteB), productionPlan(Part2, Part1), partProducedAt(Part2, SiteD).$
 $\leftarrow partProducedAt(Part1, SiteB), productionPlan(Part2, Part1), partProducedAt(Part2, SiteE).$

This example is reflected in Figure 11. For single sourcing, only one path can exist through which the full flow of parts is guided. For the multiple sourcing with n paths, the fraction of parts flow is only $1/n$ for each existing path assuming that the distribution among the paths is equal.

Singe Source



Multi Source

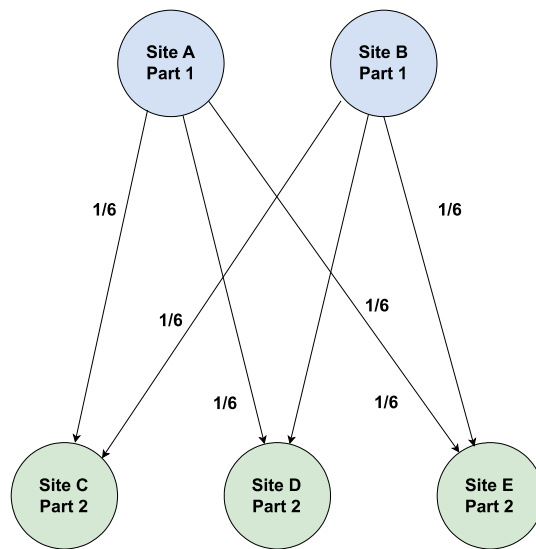


Figure 11: Single-Sourcing vs Multi-Sourcing: Multiple paths

3.3 Overview of New Features and Expected Impact

Remember that the complexity of the problem is depending on the industrial Scenario which is intended to be optimized. Therefore the design of experiments and the features to be tested will be tested on the **(baseline)** Scenario and configuration. The baseline Scenario is the double Sourcing Strategy, single batching and standard warehouse Scenario, which gives the choice between no, one or two warehouse. The baseline configuration are all facts and rules as presented in Tables 1 and 2. An overview of all new features to be tested is shown in Table 3 with the according references of each rule or group of rules in the *label* column.

The following description of the new features to be tested is split into two groups depending on the expected main impact, i.e. on the KPIs or the search tree traversal.

3.4 Main Objective Functions

Different order of weightings/priorities of the main objective functions will be tested as represented in rules **(optim-1P-2C-3T)**, **(optim-1T-2P-3C)**, **(optim-1T-2C-3P)**, **(optim-1C-2P-3T)**, **(optim-1C-2T-3P)** and **(optim-comb-Obj)**. Except for the last one, which will replace the two first *#minimize* statements, all optimization statements are part of the according configuration, but only with changing order of priorities as indicated by the naming. This is expected to have a direct quantitative impact on the KPIs. The active weighting of the optimization statements should also increase **Variability** of the answer sets by optimizing in different directions. A combined **multi-objective function** as in rule **(optim-comb-Obj)** will be tested to provoke different traversal behavior of the optimizer on its convergence paths tackling hypothesis H2. A combined function is an indirect mean to optimize two KPIs with the same priority as long as both values are normalized. This is expected to result in a different behavior of the solver than with independent optimization statements.

3.4.1 Search Tree Traversal

The active exploration of the search space is targeting to enable to prune the search space effectively and efficiently and by that increasing the probability to converge to an optimum. Only by steadily increasing the number of identified *no-goods*, the search space can be reduced to a manageable size [14]. Active traversal management can also increase variability of the solutions and reduce run-times to converge to an optimum.

For the purpose of search space pruning, the use of integrity and weak constraints to identify undesired solutions can be useful means. These are expected to increase the number of detected *no-goods* and actively trigger the main mechanism of the solver. Undesired solutions might refer to practical constraints concerning the industrial setup, for instance, that some work packages should be certainly assigned to some production sites.

On the other hand, implementation of domain knowledge by the means of integrity or weak constraints might have negative impacts as described in hypothesis H2 by excluding or penalizing solutions that could be mathematically optima, but are not desired solutions from domain experts view. Run-times until convergence may increase as well when mathematically better solutions are excluded or penalized.

The following group of integrity constraints will be tested as a set: **(minVal-IC)**. These integrity constraints ensure that minimum work-shares are allocated to certain production sites making results more valid for the end-user and expected to prune the search-space. But it can also lead to excluding mathematically better or even optimal solutions and therefore produce worse KPIs and longer run-times.

Weak constraints will be tested in two ways. One weak constraint will be replacing the **(workshare-IC)** rule from the **(baseline)** configuration: **(nb-Sites-WC)**. The weight w will be set higher than for all other optimization statements, so that over time every site will be allocated with a part.

Second approach is to test weak constraints as domain knowledge representations and compare them with domain heuristics implementations.

The strategic use of *weak constraints* as described in Section 2.4 may guide the solver and reduce the run-times. As weak constraints are light versions of integrity constraints, this might partially ease hypothesis H3 and prune the search space while also guiding the solver actively tackling hypothesis H5. The guided search into more favorable areas and rejecting solutions that are not valid or optimal is supposed to prune the search space and reduce run-times required to converge to a local optimum.

The weak constraints to be tested are **(min-hubs-on-locations)**, **(min-prefer-trucks)**, **(min-prefer-certain-locations)** and **(min-back-forth)**.

The implementation of domain heuristics is an alternative way of domain knowledge implementation. When applying a domain heuristic as described in Equation 9, it replaces the default *VSIDS* heuristics for the concerned predicates and will set according atoms to *True* when deciding on the atom. Therefore models including the pre-set domain heuristics will be found earlier.

Program excluding facts	Description	Label
<code>#heuristic producedAt(Part, P) : producableAt(Part, P). [w, true]</code>	prefer to produce part at site P	(heu-prefer-certain-locs)
<code>#heuristic producedAt(Part, P) : producedAt(Part, P), productionPlan(Super, Part), subassembly(Part), producableAt(Part, P). [w, true]</code>	prefer to produce subassemblies at same site as assemblies P	(heu-hubs-on-location)
<code>#heuristic path(Part, From, To, TM, D) : truck(TM). [w, true]</code>	prefer trucks as transportation mean	(heu-prefer-trucks)
<code>#heuristic producedAt(Super, P2) : productionPlan(SuperSuper, Super), productionPlan(Super, Part), producedAt(Part, P), producedAt(Super, P), P2 = P</code>	avoid transporting a part when it needs to be transported back	(heu-back-forth)
<code>#maximize { w, Part, Super, P : producedAt(Part, P), (choiceLvl2)(Part) }.</code>	maximize number of sites where sub- and assembly are done	(min-hubs-on-locations)
<code>#maximize { w, Part, P : productionLocation(P) }</code>	maximize production at certain sites	(min-prefer-certain-locs)
<code>#maximize { 1, Part, From, To, TM, ..., ..., truck(TM) }.</code>	maximize usage of trucks as transportation mean	(min-prefer-trucks)
<code>backAndForth(Part, SuperSuper) ← productionPlan(SuperSuper, Super), productionPlan(Super, Part), producedAt(Super, P2), producedAt(SuperSuper, P), P2 = P</code>	minimize transporting a part to be returned	(min-back-forth)
<code>#minimize { w, Part, Super, P : backAndForth(Part, SuperSuper) }</code>		
<code>#maximize { 1 @ 1 w, P : producedAt(Part, P) }.</code>	maximize number of production sites	(nb-Sites-WC)
<code>← #sum { Value, Part : producedAt(Part, P), valAdded(Part, Value), locatedIn(P,) } ≤ n</code>	work share for France,	(minVal-IC)
<code>← #sum { Value, Part : producedAt(Part, P), valAdded(Part, Value), locatedIn(P,) } ≤ n</code>	Germany,	
<code>← #sum { Value, Part : producedAt(Part, hh), valAdded(Part, Value) } ≤ m</code>	Hamburg and	
<code>← #sum { Value, Part : producedAt(Part, ths), valAdded(Part, Value) } ≤ m</code>	Toulouse with $m, n \in \mathbb{N}$	
<code>#minimize { (Value * Cost) / e @ w, Part, P : producedAt(Part, P), valAdded(Part, Value), costLoc(P, Cost) }</code>	minimize Production Costss	(optim-1P-2C-3T)
<code>#minimize { TC @ w, Part, From, To, TM : path(Part, From, To, TM, D, LT, CO2, TC) }</code>	minimize Transportation Costs	(optim-1T-2P-3C) (optim-1T-2C-3P)
<code>#minimize { CO2 @ w, Part, From, To, TM : path(Part, From, To, TM, D, LT, CO2, TC) }</code>	minimize CO ₂ emissions	(optim-1C-2P-3T) (optim-1C-2T-3P)
<code>#minimize { ((Value * Cost) + TC) / e₁ * e₂, Part, P : producedAt(Part, P), valAdded(Part, Value), costLoc(P, Cost), path(Part, From, To, TM, D, LT, CO2, TC) }</code>	minimize sum of production and transportation	(optim-comb-Obj)

Table 3: Overview of all new features to be tested .

The domain heuristics to be tested are supposed to be counterparts of the weak constraints mentioned above: **(heu-hubs-on-locations)**, **(heu-prefer-trucks)**, **(heu-prefer-certain-locations)** and **(heu-back-forth)**.

The weak constraints and domain heuristics aim at representing secured or assumed knowledge about the mathematical optimum with regards to a specific objective and are expected to find better KPIs faster for the addressed KPI, but can have adverse effects on another KPI.

(heu-prefer-certain-locations) and **(min-prefer-certain-locations)** are targeting to find better solutions quicker for the Production Costss KPIs. The optimization of Production Costss can be facilitated by guiding the search through preferred production sites, i. e. sites where the Production Costss are the lowest. Therefore this is also expected to have a positive effect on the run-times. It can lead to lower performance regarding the other KPIs though.

All other weak constraints and heuristics are addressing to improve the transportation and Co2 costs. **(min-prefer-trucks)** and **(heu-prefer-trucks)** take advantage of the fact that some transportation means are considered “standard” transportation means and therefore have the lowest Transportation Costs.

Considering the production plan, it seems reasonable that parts of the same level are transported to a possibly close site or to the same site for further assembly, the so-called hubs. Hubs can occur at different levels, but given the constraint that each site needs to produce (at least) one part in one industrial system, they should probably not be on all levels. Therefore it is chosen that sub-assembly and assembly should be produced at the same location: **(min-hubs-on-location)** and **(heu-hubs-on-location)**.

Finally the rules **(min-back-forth)** and **(min-back-forth)** are trying to avoid solutions where a part is produced in location A (previous), then sub-assembled in location B (intermediate), and then again assembled in location A.

4 Methodology

The methodology chosen and applied in this thesis is an *Experimental Quantitative Methodology*. It suits to quantify the influence of additional features and their according impact on the objectives systematically. Quantitative experimental methods enable to find patterns or associations among design parameters and the outputs and the according weighted dependencies [29].

This chapter will present how experiments will be conducted and the according data gathered and the metrics applied to measure the performance of tested means. Three metrics will be used to determine the impacts of the dedicated rules on the overall performance, i.e. the KPIs, the variability and the number of model found.

4.1 Design of Experiments

The research will be performed in two phases. In the *Main Phase* an independent analysis of each feature will be conducted to have clear visibility of the single impacts on answer sets and their associated measures, which will be presented in the next sections. After presentation and discussion of the *Main Phase* results, proposals will be made for the conclusion phase configurations to assess the conclusions made from the *Main Phase*.

The reference for all experiments are the results of the **(baseline)** configuration run, including all facts, all rules and optimization statements as shown in Table 2.

An overview of all new features to be tested is shown in Table 3.

Each heuristic and weak constraint will be added individually to the **(baseline)** configuration for simulation.

The rules **(nb-Sites-WC)** and **(minVal-IC)** and any combination of them will each replace the **(workshare-IC)** rule as they are alternative implementations of the same constraint. The workshare constraints **(minVal-IC)** and **(nb-Sites-WC)** can be combined and will be tested as well with the label **(comb-nbSites-minVal)**. So will the combination **(workshare-IC)** and **(minVal-IC)** which is labeled **(comb-baseline-minVal)** accordingly.

The optimization statements at the bottom of the Table 3 are representing the same optimization statements of the **(baseline)** configuration, only with alternative weightings of each KPIs, except for the rule **(optim-comb-Obj)**, which will replace the first two optimization statements concerning production and transportation costs.

The timeout for every run is 36.000 seconds.

Every optimization run will generate the KPIs as described in section 4.2.1. The answer sets will be analyzed regarding their variability with the method as described in section 4.2.2. And finally, the number of models will be logged for every run to compare search efficiency for the according configuration. The results of the KPIs as well as for the number of models will be compared in percentages against the reference results.

The features and configurations to be tested in phase 2 will be defined with the discussion of the phase 1 results in section 6.

4.2 Evaluation Metrics

The Evaluation Metrics described in this chapter are the parameters to measure the impact of each feature on the performance of the program. There will be two kinds of evaluation metrics:

- the first group of metrics will refer to the main objective functions of the industrial and logistics system itself and act as a direct indicator of the performance impact, i.e. the KPIs
- the second group of metrics are auxiliary performance indicators measuring the behavior of the solver and optimizer, especially with regards to its search tree traversal and act as indirect indicators, i.e. Variability and Search Efficiency

4.2.1 Key Performance Indicators of Objective Functions

Key performance indicators are all objective functions of the industrial and logistics system. They refer to the two problems of *Optimal Allocation of Part Production Across Sites* and *Optimal Routing Across Production Sites* as described in Section 1.2.1. These are:

- Production Costs: Cost for production of all parts that need to be produced. Production costs are driven by the workload per part w_p , which is fixed, and the cost factor of the dedicated production location c_s that is chosen to produce the part, which is location dependent. The specific formula for the production costs $ProdCost$ of a part is

$$ProdCost = w_p \cdot c_s$$

which are implemented by the facts **(valAdd)** and **(costLoc)**. See the according formal optimization statements and implementation of the sum of all production costs as an aggregate in the logic program in the first rule of the group of rules **(optim-baseline)**. It cumulates in the first term of the term tuple as described in the aggregate part of section 2.4 and Equation 7.

- **Transportation Costs:** Cost of transportation activities are driven by the distance D of the transport activity and the transport cost per distance factor for the transportation mean chosen. The according cost for a transportation link between two production sites that perform consecutive activities are given by facts as represented in the eighth argument in (**canBeTransp**). See the according formal optimization statements and implementation of the sum of all transportation costs as an aggregate in the logic program in the second rule of the group of rules (**optim-baseline**).
- **CO₂ emissions:** Cost of CO₂ emissions is driven by the distance of the transport activity and the transportation mean chosen similarly as for the transportation cost. Nevertheless, these two are not fully linear as CO₂ emissions and operational cost of a transportation mean are independent from each others. The according CO₂ cost for a transportation link between two production sites that perform consecutive activities is given by facts, represented as the seventh argument in (**canBeTransp**). This optimization statement of the sum of all CO₂ costs is implemented as an aggregate in the logic program in the third rule of the group of rules (**optim-baseline**).

The general reference to compare the KPIs as defined above will be the according results from the (**baseline**) run. All runs need to be confirmed by a validation run.

For the consolidated summary of all tested features, two additional trivial validation values will be determined, i.e. the best achievable production – labeled (**trivial-production**) – and transportation costs – labeled (**trivial-transportation**). Both are resulting from a (**baseline**) configuration run with 360.000 seconds timeout, except that for the optimization statements (**optim-baseline**) only a single *#minimize* statement each for production and transportation costs only will be implemented.

For the production costs even the workshare constraint (**workshare-IC**) is removed, so that parts can be assigned to the lowest cost production sites where possible without being forced to consider higher cost locations.

For transportation the trivial optimization run should result in a shortest path solution consisting of as many intrasite transports (cost zero) and transportation means that have the lowest cost per route. These two values should help to better evaluate the quality of the potential *Pareto front* in a consolidated scatter plot of the two concerned KPIs.

4.2.2 Variability Measurement

The variability of the answers sets is an indicator of how extensively the search space is explored. The more the answer sets are diverse, the more confidence can be gained that no global optimum is missed.

To measure variability of the answer sets, a similarity unit of measurement is selected that bases on the *Jaccard-Coefficient*. Originally developed by Salton & McGill in 1983 [30] in the context of electronic information retrieval, it is generally speaking a unit that measures commonality of two objects.

$$Jaccard(a, b) = \frac{|\Gamma(a) \cap \Gamma(b)|}{|\Gamma(a) \cup \Gamma(b)|} \quad (12)$$

See in Equation 12 how a *Jaccard coefficient* can be formalized. Following this, the similarity is defined by the ratio of elements that two objects a and b share versus the sum of all elements they relate to individually.

In this context it aims at comparing two answer sets that are represented as tables including the columns *Part*, *Produced At*, *Transported To* and *Transport Mean*. The row elements of the table are all transportation steps derived from the set of *path/8* predicates of the solution. As the path is determined by two decisions, i.e the allocation of parts to production sites and the choice of transportation means between two sites, the *Jaccard coefficient* will be split into two components. This could be regarded as the split into node-similarity and edge-similarity when each solution is represented as a graph.

First, the set with respect to the allocation of a part $p \in P$ to sites $s \in S$ for a model is defined as

$$S_{nodes}(p) = \{S_{nodes_1}(p), \dots, S_{nodes_n}(p)\}$$

with $S_{nodes_i}(p) = \{(p, s_i) \mid p \in P, 1 \leq i \leq n\}$ with n being the total number of sites $s \in S$ producing part $p \in P$. The corresponding set of all such sets is denoted as $S_{nodes} = \{S_{nodes}(p) \mid p \in P\}$. Likewise, the set of all sets with respect to all choices of transportation means between sites for a model (or answer set) is defined as

$$S_{edges} = \{S_{edges_1}, \dots, S_{edges_m}\}$$

with m being the number of all performed transportation activities. Accordingly, we compute the Jaccard similarity between two answer sets m_1 and m_2 following Equation 12:

$$J_{production} = \frac{|S_{nodes_{m_1}} \cap S_{nodes_{m_2}}|}{|S_{nodes_{m_1}} \cup S_{nodes_{m_2}}|}$$

$$J_{transport} = \frac{|S_{edges_{m_1}} \cap S_{edges_{m_2}}|}{|S_{edges_{m_1}} \cup S_{edges_{m_2}}|}$$

The similarity measure is applied in three ways. First, we compare the models of the first and last model of each run to determine the internal variability within a single run. Second, we aim to see the variability against a reference model, which is the last model of the **(baseline)** run to see how much variability can be achieved overall among different runs with different encoding. Finally, the configurations are compared among each other to check whether they produce diverse answer sets.

4.2.3 Search Efficiency

The *Search Efficiency* is a mean to measure how good the solver can find models for the given configuration. It will be measured by logging the number of models found until standard timeout, i.e. 36000 seconds. The figures will be presented in percentages versus the models found by the (baseline) configuration.

Another aspect of *Search Efficiency* is whether an optimum was found before timeout or not.

Finally, the grounding time is the last element of *Search Efficiency*, which will be tracked for every optimization run.

5 Results of Main Phase

This chapter presents the results of the conducted experiments described in section 4. Experiments were carried out on an AMD EPYC 7443P 24-Core Processor, 64GB RAM type DDR4-3200. The timeout was set to 360000 seconds. Note that in all Figures, only every 100th model is plotted.

5.1 Baseline Reference

Grounding took about half a minute and the first model was found after a few seconds.

The reference for all further evaluations is the **(baseline)** run with a timeout of 36000 seconds. It found more than 13.300 models with only every 100th model printed. No optimum was found.

configuration	number of models found after	
	optimum found	36.000sec
(baseline)	no	13.300 (100%)

Table 5: The number of models found for the (baseline) run

The respective KPIs for the last model are shown in Table 6 and defined as the reference with 100% each. The figures refer to the definitions from section 1.2.3, but are all factorized, so that real figures are hidden due to confidentiality.

model	Costs after 36000sec timeout		
	Production	Transport	CO ₂
last (baseline)	1.498.375 (100%)	1.322.027 (100%)	145.597 (100%)

Table 6: The reference KPIs of the (baseline) run.

Table 7 shows the percentages of the KPIs (Production Costs, Transportation Costs and CO₂ costs) for the first and the last model of the baseline run. As the last model of the baseline run does act as the main reference for all evaluations it is set to 100% as described above. The table shows that the values at the start of the run were higher with 138% for Production Costs, 131% for Transportation Costs and 126% for the CO₂ emissions.

model	% of models cost to last (baseline)		
	Production	Transport	CO ₂
last (baseline)	100	100	100
first (baseline)	115	138	131

Table 7: KPI percentages of the baseline comparing the first and the last model

Unnamed: 0	part	producedAt	transportedTo	Transport Mean	Distance	TransPRC	Co2	LeadTime
0	s6RightShell	kahramankazan	nordenham	truckOversizedLowBed	2899	13044	2261	58
1	s4UpperShell	kahramankazan	nordenham	truckOversizedLowBed	2899	13044	2261	58
2	s2UpperRightShell	troia	nordenham	truckOversizedLowBed	2311	10399	1802	46
3	s4LowerLeftShell	kahramankazan	nordenham	truckGeneralCargo	2899	10146	2055	53
4	s6UpperShell	kahramankazan	nordenham	truckGeneralCargo	2899	10146	2055	53
5	s5LeftShell	oberpfaffenhofen	nordenham	truckOversizedLowBed	780	3511	608	16
6	section7	meaulte	montoindeBretagne	oversizedBelugaXL	463	14824	22916	1
7	s4LowerRightShell	kahramankazan	hamburg	truckLongDistance	2841	9944	2015	52
8	section4	nordenham	hamburg	oversizedUnderDeckRoRoShip	153	978	24	6
9	section3	nordenham	hamburg	oversizedUnderDeckRoRoShip	153	978	24	6
10	section7	nordenham	hamburg	oversizedUnderDeckRoRoShip	153	978	24	6
11	s4LowerLeftShell	kahramankazan	hamburg	truckGeneralCargo	2841	9944	2015	52
12	s5LowerRightShell	kahramankazan	hamburg	truckGeneralCargo	2841	9944	2015	52
13	s2UpperRightShell	troia	hamburg	truckOversizedLowBed	2368	10658	1847	47
14	section7	meaulte	hamburg	oversizedBelugaXL	631	20207	31239	1
15	section67	montoindeBretagne	toulouse	oversizedBelugaXL	402	15745	24341	1
16	section1	hamburg	toulouse	oversizedBelugaXL	402	15745	24341	1
17	section2345	montoindeBretagne	toulouse	oversizedBelugaXL	402	15745	24341	1
18	section67	hamburg	toulouse	oversizedBelugaXL	1266	40507	62621	2
19	section1	hamburg	toulouse	oversizedBelugaXL	1266	40507	62621	2
20	section2345	hamburg	toulouse	oversizedBelugaXL	1266	40507	62621	2
21	section5	hamburg	hamburg	intrastieTransport	0	0	0	0
22	section2345	hamburg	hamburg	intrastieTransport	0	0	0	0
23	section3	hamburg	hamburg	intrastieTransport	0	0	0	0
24	section1	hamburg	hamburg	intrastieTransport	0	0	0	0
25	section6	hamburg	hamburg	intrastieTransport	0	0	0	0
26	s3RightShell	hamburg	hamburg	intrastieTransport	0	0	0	0
27	section4	hamburg	hamburg	intrastieTransport	0	0	0	0
28	section2	hamburg	hamburg	intrastieTransport	0	0	0	0
29	section67	hamburg	hamburg	intrastieTransport	0	0	0	0
30	s3RightShell	nordenham	hamburg	intrastieTransport	0	0	0	0
31	s6UpperShell	nola	(naples)Harbor,nordenham	(truckGeneralCargo,truckLongDistance)	1902	6656	1349	35
32	s2LowerRightShell	nola	(naples)Harbor,nordenham	(truckOversizedStandardBed,truckGeneralCargo)	1902	6684	1351	35
33	s3LeftShell	oberpfaffenhofen	(marseille)Harbor,hamburg	(truckOversizedLowBed,truckOversizedLowBed)	2339	10523	1824	47
34	section2	nordenham	(montoir)Harbor,montoindeBretagne)	(oversizedUnderDeckRoRoShip,largeOversizedRoadTransport)	2599	17623	481	104
35	section3	nordenham	(montoir)Harbor,montoindeBretagne)	(oversizedUnderDeckRoRoShip,largeOversizedRoadTransport)	2599	17623	481	104
36	section7	nordenham	(montoir)Harbor,montoindeBretagne)	(oversizedUnderDeckRoRoShip,largeOversizedRoadTransport)	2599	17623	481	104
37	s6LeftShell	nordenham	(montoir)Harbor,nordenham)	(truckOversizedLowBed,truckOversizedLowBed)	3093	13920	2413	62
38	section67	montoindeBretagne	(montoir)Harbor,hamburg)	(largeOversizedRoadTransport,oversizedUnderDeckRoRoShip)	2678	18111	492	107
39	section2345	montoindeBretagne	(montoir)Harbor,hamburg)	(largeOversizedRoadTransport,oversizedUnderDeckRoRoShip)	2678	18111	492	107
40	section67	hamburg	(mobile)Harbor,mobile)	(oversizedUnderDeckRoRoShip,largeOversizedRoadTransport)	9219	60882	1599	369
41	s4UpperLeftShell	kahramankazan	(dunkerque)Harbor,nordenham)	(truckGeneralCargo,truckLongDistance)	2894	13313	2497	69
42	s3LeftShell	rochefort	(dunkerque)Harbor,nordenham)	(truckOversizedLowBed,truckOversizedLowBed)	1398	6293	1091	28
43	s3UpperShell	kahramankazan	(dunkerque)Harbor,nordenham)	(truckLongDistance,seaLargeContainerShip)	3763	11926	2278	81
44	s4LeftShell	rochefort	(gruenedich)Harbor,hamburg)	(truckOversizedLowBed,truckOversizedLowBed)	1351	6083	1055	27
45	s4UpperShell	kahramankazan	(gruenedich)Harbor,hamburg)	(truckOversizedStandardBed,truckOversizedStandardBed)	2865	12897	2236	57
46	s4LowerRightShell	kahramankazan	(gruenedich)Harbor,nordenham)	(truckLongDistance,truckLongDistance)	2952	10333	2094	54
47	s4UpperRightShell	kahramankazan	(gruenedich)Harbor,hamburg)	(truckLongDistance,truckShortDistance)	2865	10013	2032	52
48	s3UpperLeftShell	kahramankazan	(gruenedich)Harbor,hamburg)	(truckLongDistance,truckLongDistance)	2865	10030	2032	52
49	s2LowerLeftShell	kahramankazan	(gruenedich)Harbor,hamburg)	(truckLongDistance,truckLongDistance)	2865	10030	2032	52
50	s2LowerRightShell	troia	(gruenedich)Harbor,nordenham)	(truckGeneralCargo,truckGeneralCargo)	2470	8645	1752	45
51	s2LowerLeftShell	troia	(gruenedich)Harbor,nordenham)	(truckGeneralCargo,truckGeneralCargo)	2383	8342	1690	40
52	s6LeftShell	nola	(gruenedich)Harbor,hamburg)	(truckOversizedLowBed,truckOversizedLowBed)	1872	8428	1461	37
53	s6RightShell	kahramankazan	(gruenedich)Harbor,hamburg)	(truckOversizedLowBed,truckOversizedLowBed)	2865	12897	2236	57
54	s3LowerRightShell	kahramankazan	(gruenedich)Harbor,nordenham)	(truckOversizedLowBed,truckOversizedStandardBed)	2865	13287	2203	59
55	s4UpperRightShell	kahramankazan	(gruenedich)Harbor,nordenham)	(truckGeneralCargo,truckLongDistance)	2952	10333	2094	54
56	s3LowerLeftShell	kahramankazan	(gruenedich)Harbor,nordenham)	(truckGeneralCargo,truckShortDistance)	2952	10247	2094	54
57	s3RightShell	nordenham	(hamburg)Harbor,hamburg)	(oversizedUnderDeckRoRoShip,kugelbake)	167	1272	42	7
58	section6	nordenham	(hamburg)Harbor,hamburg)	(oversizedUnderDeckRoRoShip,kugelbake)	167	1272	42	7
59	section5	nordenham	(hamburg)Harbor,hamburg)	(oversizedUnderDeckRoRoShip,kugelbake)	167	1272	42	7
60	s3RightShell	hamburg	(hamburg)Harbor,nordenham)	(oversizedUnderDeckRoRoShip,kugelbake)	167	4249	266	13
61	s3UpperRightShell	kahramankazan	(hamburg)Harbor,nordenham)	(truckOversizedLowBed,oversizedUnderDeckRoRoShip)	3000	13799	2242	63
62	s4UpperRightShell	kahramankazan	(hamburg)Harbor,hamburg)	(truckOversizedLowBed,oversizedUnderDeckRoRoShip)	2853	13459	2234	61
63	s6LowerShell	kahramankazan	(hamburg)Harbor,nordenham)	(truckOversizedLowBed,oversizedUnderDeckRoRoShip)	2979	13406	2323	60
64	s6LowerShell	kahramankazan	(hamburg)Harbor,hamburg)	(truckOversizedLowBed,oversizedUnderDeckRoRoShip)	2853	12857	2219	57
65	s6LeftShell	kahramankazan	(hamburg)Harbor,hamburg)	(truckOversizedLowBed,oversizedUnderDeckRoRoShip)	2979	13406	2323	60
66	s6LeftShell	kahramankazan	(hamburg)Harbor,hamburg)	(truckOversizedLowBed,oversizedUnderDeckRoRoShip)	2853	12859	2225	57
67	section2	nordenham	(hamburg)Harbor,hamburg)	(kugelbake,kugelbake)	167	4451	281	14
68	s3UpperLeftShell	kahramankazan	(hamburg)Harbor,nordenham)	(truckLongDistance,seaLargeContainerShip)	2979	10426	2113	54
69	s3LowerLeftShell	kahramankazan	(hamburg)Harbor,nordenham)	(truckLongDistance,seaLargeContainerShip)	3000	10176	2026	58
70	s3UpperShell	kahramankazan	(hamburg)Harbor,nordenham)	(truckLongDistance,truckGeneralCargo)	2979	10426	2113	54
71	s2UpperShell	kahramankazan	(hamburg)Harbor,nordenham)	(truckLongDistance,truckShortDistance)	2980	10320	2113	54
72	s3UpperRightShell	troia	(hamburg)Harbor,hamburg)	(truckOversizedLowBed,oversizedUnderDeckRoRoShip)	2387	10762	1856	48
73	s3UpperRightShell	troia	(hamburg)Harbor,nordenham)	(truckOversizedLowBed,oversizedUnderDeckRoRoShip)	2513	11311	1960	50
74	s4LowerShell	kahramankazan	(hamburg)Harbor,hamburg)	(truckGeneralCargo,kugelbake)	2853	10216	2033	53
75	s3LowerLeftShell	kahramankazan	(hamburg)Harbor,hamburg)	(truckGeneralCargo,truckLongDistance)	2853	9986	2023	52
76	s3UpperShell	kahramankazan	(hamburg)Harbor,hamburg)	(truckGeneralCargo,seaLargeContainerShip)	2853	9964	2017	52
77	s3UpperShell	kahramankazan	(hamburg)Harbor,hamburg)	(truckGeneralCargo,seaLargeContainerShip)	2853	9977	2023	52
78	s2LowerLeftShell	kahramankazan	(hamburg)Harbor,nordenham)	(truckGeneralCargo,truckLongDistance)	2980	10430	2113	54
79	s3UpperShell	sacheon	(sacheon)Harbor,hamburg)	(truckShortDistance,seaLargeContainerShip)	2980	4057	1747	1111
80	s4LowerShell	kahramankazan	(naples)Harbor,marseille)Harbor,nordenham)	(truckGeneralCargo,seaLargeContainerShip,truckLongDistance)	4176	12840	2494	96
81	s6UpperShell	nola	(naples)Harbor,marseille)Harbor,hamburg)	(truckLongDistance,seaLargeContainerShip,truckLongDistance)	3301	6276	1074	62
82	s6UpperShell	kahramankazan	(sacheon)Harbor,marseille)Harbor,hamburg)	(truckGeneralCargo,seaLargeContainerShip,truckLongDistance)	7793	22789	4118	189
83	s4LowerRightShell	sacheon	(sacheon)Harbor,marseille)Harbor,nordenham)	(truckShortDistance,seaLargeContainerShip,seaLargeContainerShip)	3098	44531	9841	1236
84	s4UpperRightShell	sacheon	(sacheon)Harbor,marseille)Harbor,hamburg)	(truckShortDistance,seaLargeContainerShip,truckGeneralCargo)	27020	41837	2610	1059
85	s4UpperRightShell	sacheon	(sacheon)Harbor,marseille)Harbor,nordenham)	(truckGeneralCargo,seaLargeContainerShip,truckGeneralCargo)	26962	41649	2569	1040
86	s4UpperLeftShell	sacheon	(sacheon)Harbor,montoir)Harbor,hamburg)	(truckShortDistance,seaLargeContainerShip,seaLargeContainerShip)	28257	40715	1775	1130
87	s2UpperLeftShell	sacheon	(sacheon)Harbor,shanghai)Harbor,hamburg)	(truckGeneralCargo,seaLargeContainerShip,seaLargeContainerShip)	27996	40351	1759	1119
88	s2UpperLeftShell	sacheon	(sacheon)Harbor,shanghai)Harbor,nordenham)	(truckGeneralCargo,seaLargeContainerShip,seaLargeContainerShip)	27916	40237	1754	1116
89	s3UpperShell	toulouse	(porto)Harbor,hamburg)Harbor,hamburg)	(truckLongDistance,seaLargeContainerShip,truckLongDistance)	4321	8473	975	149
90	s4UpperLeftShell	kahramankazan	(dunkerque)Harbor,hamburg)Harbor,hamburg)	(truckLongDistance,seaLargeContainerShip,truckLongDistance)	3865	12095	2290	85
91	s3UpperLeftShell	kahramankazan	(dunkerque)Harbor,hamburg)Harbor,hamburg)	(truckLongDistance,seaLargeContainerShip,truckGeneralCargo)	3865	12095	2290	85
92	s3UpperShell	toulouse	(dunkerque)Harbor,hamburg)Harbor,nordenham)	(truckLongDistance,seaLargeContainerShip,seaLargeContainerShip)	1818	4607	738	52
93	s3LowerLeftShell	kahramankazan	(dunkerque)Harbor,hamburg)Harbor,hamburg)	(truckGeneralCargo,seaLargeContainerShip,truckLongDistance)	3865	12095	2290	85
94	s4LowerLeftShell	sacheon	(sacheon)Harbor,hamburg)Harbor,hamburg)	(truckShortDistance,seaLargeContainerShip,kugelbake)	27915	40329	1764	1112
95	s4LowerRightShell	sacheon	(sacheon)Harbor,hamburg)Harbor,hamburg)	(truckShortDistance,seaLargeContainerShip,truckGeneralCargo)	27815	40099	1754	1111
96	s3UpperLeftShell	sacheon	(sacheon)Harbor,hamburg)Harbor,nordenham)	(truckShortDistance,seaLargeContainerShip,truckLongDistance)	27942	40543	1844	1113
97	s3UpperLeftShell	sacheon	(sacheon)Harbor,hamburg)Harbor,hamburg)	(truckGeneralCargo,seaLargeContainerShip,truckShortDistance)	27815	40104	1754	1111
98	s3LowerLeftShell	sacheon	(sacheon)Harbor,hamburg)Harbor,nordenham)	(truckShortDistance,seaLargeContainerShip,truckShortDistance)	27942	40433	1844	1113
99	s2LowerLeftShell	sacheon	(sacheon)Harbor,hamburg)Harbor,hamburg)	(truckGeneralCargo,seaLargeContainerShip,oversizedUnderDeckRoRoShip)	27815	40141	1749	1111
100	s2UpperShell	sacheon	(sacheon)Harbor,hamburg)Harbor,nordenham)	(truckGeneralCargo,seaLargeContainerShip,truckShortDistance)	27942	40447	1844	1113
101	s2UpperShell	sacheon	(sacheon)Harbor,hamburg)Harbor,hamburg)	(truckGeneralCargo,seaLargeContainerShip,oversizedUnderDeckRoRoShip)	27815	40141	1749	1111
102	s3LowerLeftShell	sacheon	(sacheon)Harbor,hamburg)Harbor,hamburg)	(truckGeneralCargo,seaLargeContainerShip,truckGeneralCargo)	27815	40113	1754	1111
103	s4LowerShell	sacheon	(sacheon)Harbor,gibraltar)Harbor,hamburg)	(truckGeneralCargo,seaLargeContainerShip,seaLargeContainerShip)	28423	40967	1786	1136
104	s2UpperLeftShell	sacheon	(sacheon)Harbor,gibraltar)Harbor,nordenham)	(truckShortDistance,seaLargeContainerShip,seaLargeContainerShip)	28344	40839	1781	1123
105	s4LowerShell	sacheon	(sacheon)Harbor,gibraltar)Harbor,nordenham)	(truckGeneralCargo,seaLargeContainerShip,seaLargeContainerShip)	27803	40074	1747	1112
106	s4LowerLeftShell	sacheon	(sacheon)Harbor,porto)Harbor,hamburg)	(truckGeneralCargo,seaLargeContainerShip,truckLongDistance)	27061	43954	3255	1030
107	s3UpperShell	sacheon	(sacheon)Harbor,porto)Harbor,nordenham)	(truckGeneralCargo,seaLargeContainerShip,seaLargeContainerShip)	27803	40074	1747	1112
108	s3LowerLeftShell	sacheon	(sacheon)Harbor,porto)Harbor,nordenham)	(truckOversizedLowBed,seaLargeContainerShip,seaLargeContainerShip)	27803	40083	1748	1112
109	s4UpperLeftShell	sacheon	(sacheon)Harbor,lianjin)Harbor,nordenham)	(truckGeneralCargo,seaLargeContainerShip,seaLargeContainerShip)	29472	42478	1850	1178
110	s4LowerLeftShell	sacheon	(sacheon)Harbor,casablanca)Harbor,nordenham)	(truckGeneralCargo,seaLargeContainerShip,seaLargeContainerShip)	28065	40452	1764	1122
111	s6RightShell	sacheon	(sacheon)Harbor,marseille)Harbor,hamburg)	(truckOversizedLowBed,oversizedUnderDeckRoRoShip,truckOversizedLowBed)	27020	170214	5209	1052
112	section2	hamburg	(hamburg)Harbor,montoir)Harbor,montoindeBretagne)	(kugelbake,oversizedUnderDeckRoRoShip,largeOversizedRoadTransport)	2693	18424	512	108
113	section4	nordenham	(hamburg)Harbor,montoir)Harbor,montoindeBretagne)	(oversizedUnderDeckRoRoShip,oversizedUnderDeckRoRoShip,largeOversizedRoadTransport)	2840	19164	520	113
114	section4	hamburg	(hamburg)Harbor,montoir)Harbor,montoindeBretagne)	(oversizedUnderDeckRoRoShip,oversizedUnderDeckRoRoShip,largeOversizedRoadTransport)	2693	18222	497	107
115	section6	hamburg	(hamburg)Harbor,montoir)Harbor,montoindeBretagne)	(oversizedUnderDeckRoRoShip,oversizedUnderDeckRoRoShip,largeOversizedRoadTransport)	2693	18222	497	107
116	section3	hamburg	(hamburg)Harbor,montoir)Harbor,montoindeBretagne)	(oversizedUnderDeckRoRoShip,oversizedUnderDeckRoRoShip,largeOversizedRoadTransport)	2693	18222	497	107
117	section5	nordenham	(hamburg)Harbor,montoir)Harbor,montoindeBretagne)	(kugelbake,oversizedUnderDeckRoRoShip,largeOversizedRoadTransport)	2840	22343	759	120
118	s4UpperShell	sacheon	(sacheon)Harbor,montoir)Harbor,hamburg)	(truckGeneralCargo,oversizedUnderDeckRoRoShip,truckGeneralCargo)	26882	168228	5024	1047
119	section6	nordenham	(dunkerque)Harbor,montoir)Harbor,montoindeBretagne)	(oversizedUnderDeckRoRoShip,oversizedUnderDeckRoRoShip,largeOversizedRoadTransport)	3203	21489	877	128
120	section5	hamburg	(dunkerque)Harbor,montoir)Harbor,montoindeBretagne)	(oversizedUnderDeckRoRoShip,oversizedUnderDeckRoRoShip,largeOversizedRoadTransport)	3291	22052	592	132
121	section1	hamburg	(hamburg)Harbor,mobile)Harbor,mobile)	(kugelbake,oversizedUnderDeckRoRoShip,largeOversizedRoadTransport)	9233	61176	1617	371
122	section2345	hamburg	(hamburg)Harbor,mobile)Harbor,mobile)	(oversizedUnderDeckRoRoShip,oversizedUnderDeckRoRoShip,largeOversizedRoadTransport)	9233	60974	1602	370
123	section67	montoindeBretagne	(montoir)Harbor,mobile)Harbor,mobile)	(largeOversizedRoadTransport,oversizedUnderDeckRoRoShip,largeOversizedRoadTransport)	8399	56605	1531	337
124	section1	nola	(montoir)Harbor,mobile)Harbor,mobile)	(largeOversizedRoadTransport,oversizedUnderDeckRoRoShip,largeOversizedRoadTransport)	8399	56605	1531	337
125	section2345	montoindeBretagne	(montoir)Harbor,mobile)Harbor,mobile)	(largeOversizedRoadTransport,oversizedUnderDeckRoRoShip,largeOversizedRoadTransport)	8399	56605	1531	337
126	s2LowerRightShell	kahramankazan	(dunkerque)Harbor,hamburg)Harbor,hamburg)	(truckGeneralCargo,oversizedUnderDeckRoRoShip,seaLargeContainerShip)	2469	10661	1366	60
127	s2UpperLeftShell	kahramankazan	(dunkerque)Harbor,hamburg)Harbor,nordenham)	(truckGeneralCargo,oversizedUnderDeckRoRoShip,seaLargeContainerShip)	4012	15733	2361	91
128	section1	sacheon	(montoir)Harbor,hamburg)Harbor,hamburg)	(largeOversizedRoadTransport,oversizedUnderDeckRoRoShip,kugelbake)	2693	18404	510	108
129	s3RightShell	sacheon	(sacheon)Harbor,gibraltar)Harbor,nordenham)	(truckOversizedLowBed,oversizedUnderDeckRoRoShip,truckOversizedLowBed)	27011	16778	5996	1024
130	s2UpperRightShell	sacheon	(sacheon)Harbor,gibraltar)Harbor,hamburg)	(truckOversizedLowBed,oversizedUnderDeckRoRoShip,truckOversizedLowBed)	27011	167997	6041	1027
131	s4UpperShell	sacheon	(sacheon)Harbor,gibraltar)Harbor,nordenham)	(truckOversizedStandardBed,oversizedUnderDeckRoRoShip,truckOversizedStandardBed)	27011	16778	5996	1026
132								

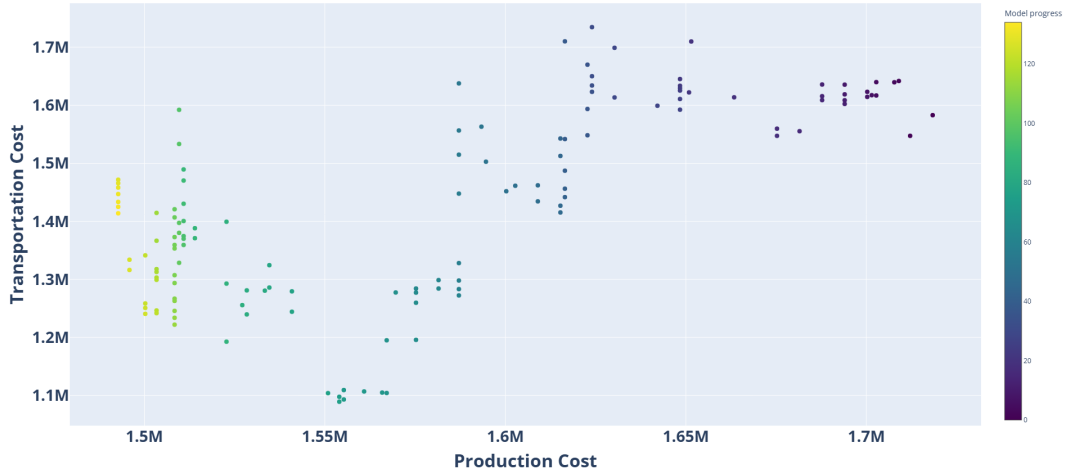


Figure 12: Evolution of the Production Costs with respect to the Transportation Costs within the (baseline) run

Figure 12 does visualize the evolution of the Production Costs with respect to the Transportation Costs. Each dot represents a model and the progress is shown by the colors of the dots, moving from dark lila for the first model to bright yellow for the last. A *Pareto front* is indicated as described in Section 1.2.3 and Figure 5. The last models are those with the lowest production cost from the supposed *Pareto front*, but not the lowest Transportation Costs that were found in earlier iterations.

Figure 12 on the other hand shows the evolution of the Production Costs with respect to the CO₂ costs with a similar behavior. The models move from the upper right corner to the lower left and a Pareto Front is indicated, but less obvious for both KPIs.

The last scatter plot in Figure 14 visualizes the evolution of Transportation Costs with respect to the CO₂ costs. While there is still a tendency of dots moving from the upper right corner to the lower left, there is no apparent *Pareto front* and the last models are clearly not in the area of an expected *Pareto optimum* with regards to the two KPIs.

Finally, Figure 15 presents how the *Jaccard similarity* values evolve from the first to the last model within the baseline run. The x-axis is the index of the model and the y-axis is the *Jaccard similarity* value, which is split into production (orange) and transportation (green) similarity. The figure shows the according similarity values of each model compared with the last (best) model.

The first model of the baseline run has similarities with the last model between 0.16 for transportation and 0.23 for production similarity. Similarity values steadily go up until finally reaching 1.0 for both values.

The models are represented by the *path/8* predicate as described in rule **(choicePath)**. Find

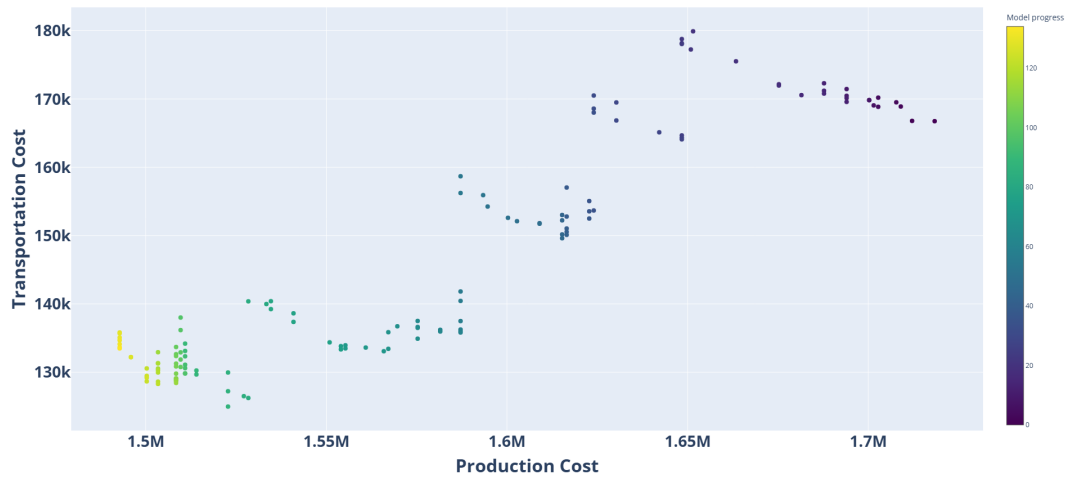


Figure 13: Evolution of the Production Costs with respect to the CO₂ costs within the (base-line) run

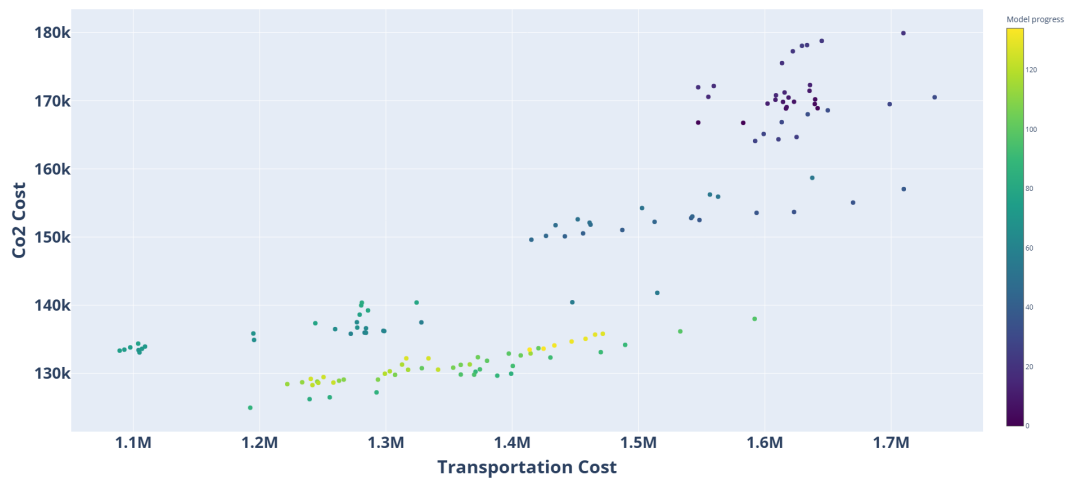


Figure 14: Evolution of the Transportation Costs with respect to the CO₂ costs within the (baseline) run

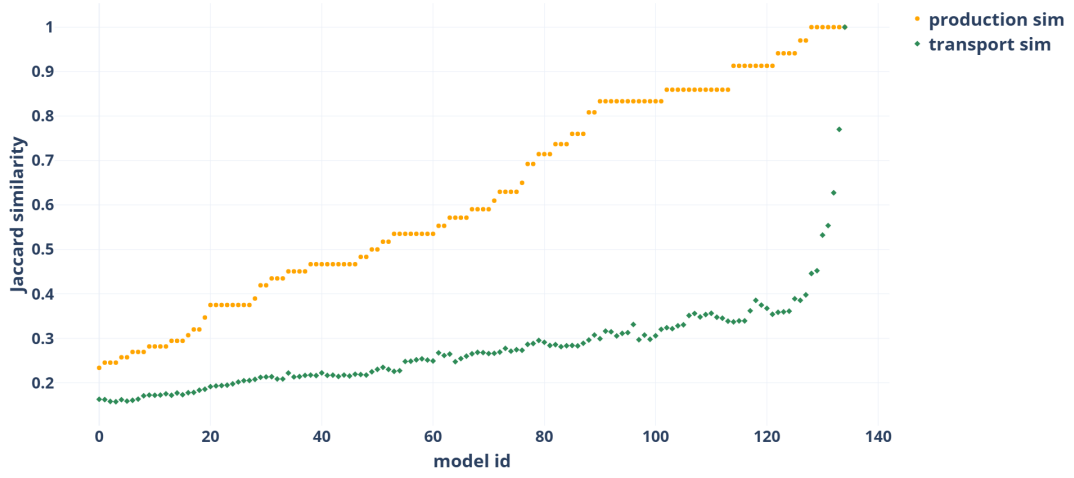
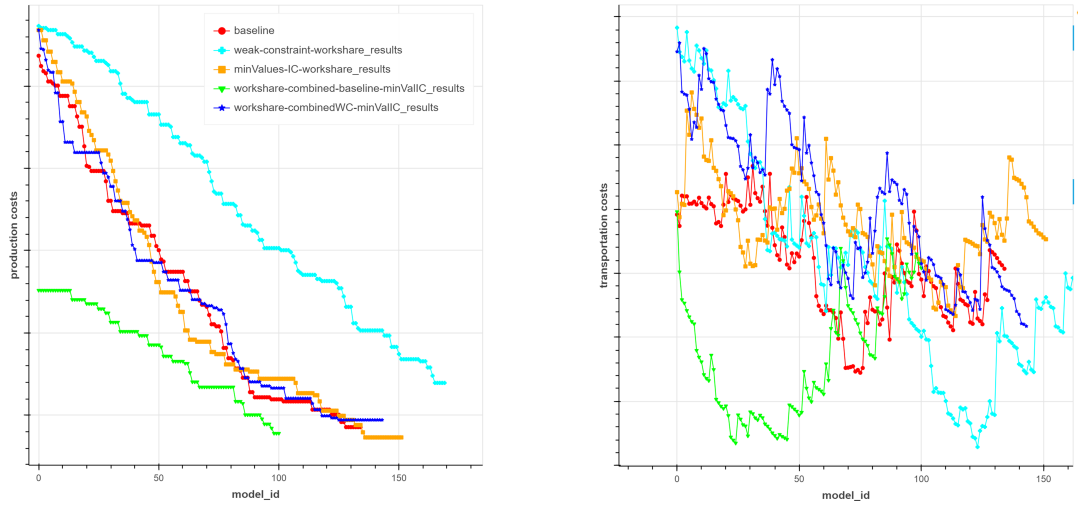


Figure 15: Evolution of the similarity values within the baseline run.

5.2 Workshare Constraint

5.2.1 KPIs of Workshare Constraint Representation



(a) Evolution of the Production Costs for the different (workshare) implementations

(b) Evolution of the Transportation Costs for the different (workshare) implementations

Figure 16: Evolution of the KPIs for the different (workshare) implementations

Figure 16 shows how the models' KPIs evolved over time. The x-axis refers to the model id (every 100th model) and the y-axis refers to the respective KPI. Regarding the Production Costs, the baseline run, the (min-Val-IC) run and the (combined nbSites-WC/minVal-IC) run progress very closely and produce nearly the same number of models while the (combined baseline/minVal-IC) starts with the far best Production Costs and remains steadily below all other runs, but produces less models (see Figure 10). All configurations produce continuously decreasing

(workshare) implementation	% of models cost to last (baseline)		
	Production	Transport	CO ₂
last (minVal-IC)	99%	107%	118%
last (nbSites-WC)	102%	106%	110%
last (combined baseline/minVal-IC)	100%	100%	205%
last (combined nbSites-WC/minVal-IC)	100%	87%	112%

Table 8: The KPI percentages of each heuristic in relation to the last (**baseline**) model. The best results are highlighted in gray.

Production Costs and end up in similar production cost values.

The behavior concerning the Transportation Costs differs strongly from the relatively homogeneous evolutions of the Production Costs. All configurations have ups and downs throughout the runs, but finally reach similar transportation cost values. The (combined-baseline-minVal) configuration does produce lower values compared with all other configurations from the beginning before arriving in the same area as the others towards the end of the ten hour run.

The CO₂ costs evolve similarly for all configurations, except for (combined-baseline-minVal) with steadily higher values, ending up at 205% above the baseline configuration (see Table 8).

Figure 17 is exemplarily visualizing all workshare representations on a scatter plot showing the Production Costs with regards to the Transportation Costs. This view emphasizes the supposed *Pareto front* from Figure 12 by producing diverse models with diverse KPIs supporting the visibility of a *Pareto front*. The variability of the KPIs among the different workshare representations indicates variability of answers sets as only differences in the models can lead to differences of the KPIs.

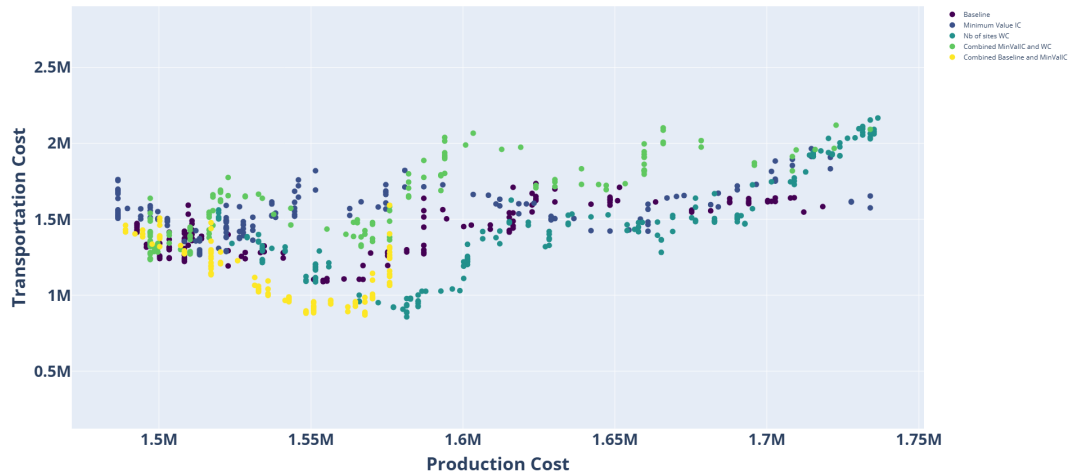


Figure 17: Evolution of the Production Costs with respect to the Transportation Costs within the (baseline) run

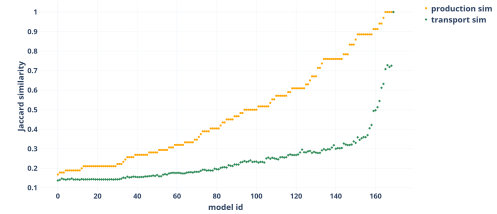
The direct measurement of the variability is subject to the next subsection.

5.2.2 Variability

The similarities within the runs are shown in four graphs, Figures 18a, 18b, 19a and 19b. Most of them are evolving similarly, with the production similarity starting slightly above the transportation similarity around 0.2, steadily remaining upon the transportation similarity until finally reaching 1. The transportation similarities are increasing slower for most of the run, but having a steep increase for the last models until reaching 1, too. In Figure 19, 19b the behavior of (comb-baseline-minVal) run differs from the others. It starts with a higher production similarity value of 0.6 already.

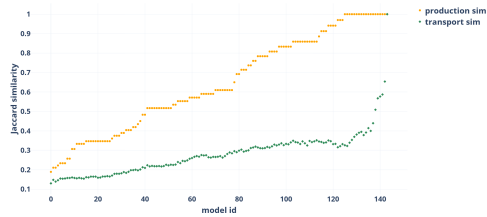


(a) Evolution of the similarities within the (minVal) configuration run

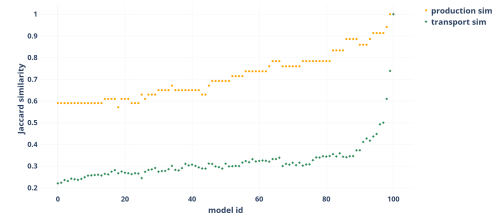


(b) Evolution of the similarities within the (nbSites-WC) configuration run

Figure 18: Similarity evolution of two configurations



(a) Evolution of the similarities within the combined (nbSites-WC) and (minVal-IC) configuration run



(b) Evolution of the similarities within the combined (baseline) and (minVal-IC) configuration run

Figure 19: Similarity evolution of two combined configurations

Figure 9 is providing an overview of all *Jaccard similarities* among the applied workshare configurations for both production (p) and transportation (t) similarities. Production similarities vary between 0.375 to 0.69 while transportation similarities range between 0.16 and 0.3.

Heuristic	(minVal-IC)		(nbSites-WC)		(baseline/minVal-IC)		(nbSites-WC/minVal-IC)	
	p	t	p	t	p	t	p	t
(baseline)	0.69	0.29	0.5	0.24	0.53	0.2	0.63	0.3
(minVal-IC)	1	1	0.55	0.24	0.51	0.18	0.71	0.29
(nbSites-WC)			1	1	0.375	0.16	0.55	0.25
(baseline/minVal-IC)					1	1	0.48	0.17
(nbSites-WC/minVal-IC)							1	1

Table 9: Similarity comparison for all (workshare) implementation options for production sites (p) and transportation similarities (t).

5.2.3 Search Efficiency

(workshare) implementation	number of models found after	
	optimum found	36.000sec
(minVal-IC)	no	113%
(nbSites-WC)	no	126%
(comb. baseline/minVal-IC)	no	75%
(comb. minVal-IC/nbSites-WC)	no	107%

Table 10: The number of models found for the different (workshare) implementations

Table 10 shows the number of models found until timeout for all different workshare representations. The **(nbSites-WC)** configuration was able to find 26% more models compared to the **(baseline)** while the **(minVal-IC)** and **(comb. minVal-IC/nbSites-WC)** configurations still found 13% or 7% respectively more models. Only **(comb. baseline/minVal-IC)** found less, namely 25% less models than the **(baseline)**.

5.3 Domain Heuristics

5.3.1 KPI evolution

Figure 20 gives an overview of the evolution of the KPIs of the heuristics runs.

For the Production Costs, the **(baseline)**, the **(heu-reduce-back-forth)** and the **(heu-prefer-certain-locations)** configurations proceed closely with the **(heu-prefer-certain-locations)** heuristic achieving better results shortly after start and the **(baseline)** finding better solutions quicker towards the end of the run. The **(heu-hubs-on-locations-sub-assembly)** starts with significantly better values and remains with better values until it finally ends up with similar values as all other configurations, except the **(heu-prefer-trucks)** heuristics which started with relatively good values, but finishes worst with just minor improvements compared with the first model. All heuristics end with the same result as the **(baseline)** run or worse (see Figure 11).

The Transportation Costs on the other hand are all better compared to the **(baseline)** in the end for the applied heuristics, except for the **(heu-reduce-back-forth)** configuration. All heuristics improve the transportation cost values over time, but only the **(heu-prefer-trucks)** configuration ends up slightly worse than at the start, which is by far the lowest transportation cost at the beginning. The **(heu-hubs-on-location-sub-assembly)** heuristics achieves the best improvements right away from the start and finally ends at 92% compared with the baseline, only outperformed by the **(heu-prefer-trucks)** configuration with 88% versus the baseline (compare Table 11). The CO₂ costs evolution leads to three observations: First, almost all heuristics have different starting points. Second, except for the **(heu-prefer-trucks)** configuration (174%, see table11) all end in the same area, around 100% of the baseline.

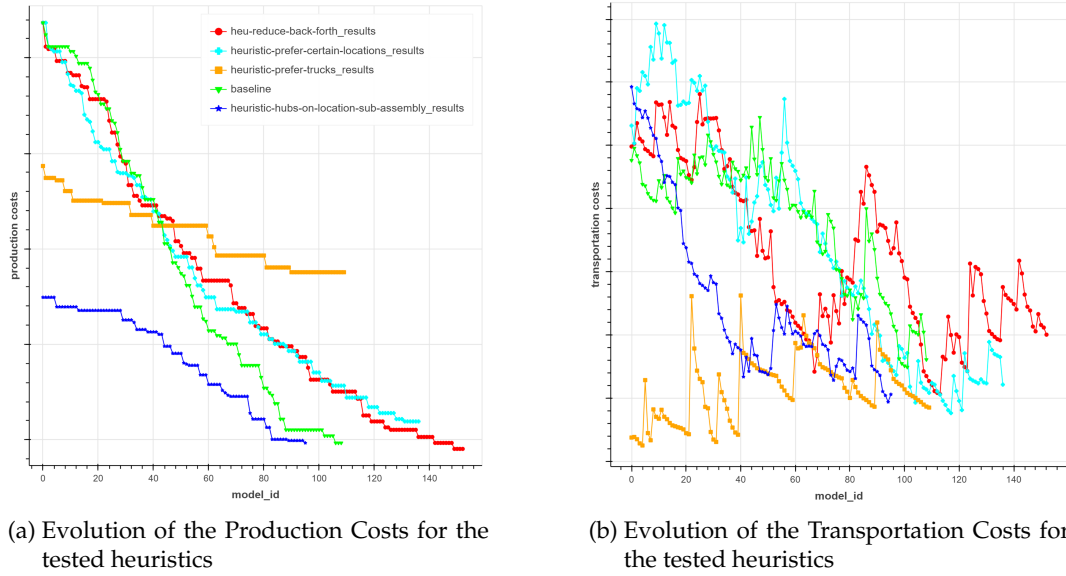


Figure 20: Overview of the resulting KPIs from the domain heuristics runs

And third, while starting worst, the (heu-hubs-on-location-sub-assembly) heuristic outpaces all others with steep improvements before reaching 3000 models.

5.3.2 Variability

Concerning the *Jaccard similarities* among all heuristics, diversity is higher than for the (**workshare**) implementations, especially for the production similarities with values between 0.2 and 0.5. The transportation variability is again higher with values of 0.15 to 0.23. See Table 12 for the precise values.

5.3.3 Search Efficiency

Grounding took between 0.37 and 0.38 seconds. The search efficiency is reflected in table 13. The (heu-hubs-on-locations) and (heu-prefer-trucks) configurations only find 71% and 82% of the number of models of the baseline run. (heu-prefer-certain locations) finds slightly more and (heu-reduce-back-forth) find even 14% more than the baseline. No optimum was found any configuration.

Heuristic	% of models cost to last (baseline)		
	Production	Transport	CO ₂
last (heu-reduce-back-forth)	100%	106%	102%
last (heu-prefer-certain-locations)	100%	94%	98%
last (heu-prefer-trucks)	106%	88%	174%
last (heu-hubs-on-location)	100%	92%	105%

Table 11: The KPI percentages of each heuristic in relation to the last (**baseline**) model. The best results are highlighted in gray.

Heuristic	(reduce-back-forth)		(hubs-on-locations)		(prefer-certain-locations)		(prefer-trucks)	
	p	t	p	t	p	t	p	t
(baseline)	0.65	0.27	0.52	0.2	0.61	0.3	0.2	0.15
(reduce-back-forth)	1	1	0.55	0.24	0.61	0.27	0.22	0.16
(hubs-on-location)			1	1	0.43	0.19	0.15	0.11
(prefer-certain-locations)					1	1	0.2	0.15
(prefer-trucks)							1	1

Table 12: Similarity comparison for all heuristics, with respect to the production sites (p) and transportation means (t).

heuristic	number of models found after	
	optimum found	36.000sec
(heu-reduce-back-forth)	no	114%
(heu-hubs-on-location)	no	71%
(heu-prefer-certain-locations)	no	102%
(heu-prefer-trucks)	no	82%

Table 13: The number of models found for the different heuristics

5.4 Weak constraints

5.4.1 KPI evolution

The evolution of the KPIs can be seen in Figure 21.

The Production Costs evolution is similar for all weak constraints and they achieve values 99% and 102% compared with the (baseline) with (min-prefer-certain-locations) being the one configuration performing better than the (baseline) and finding more models before timeout (+32% versus (baseline)) while all other configurations find less (see table 16). (min-reduce-back-forth) has a lower starting point than the others, but quickly moves in the same areas as the other configurations.

The Transportation Costs start with similar values for all configurations and end close to the baseline for (min-prefer-trucks) and (min-hubs-on-locations). (min-reduce-back-forth) quickly improves and finishes best with 93% versus (baseline). (min-prefer-certain-locations) performs worst, ending at 121% versus (baseline).

CO₂ values start and evolve similarly for all configurations, except for (min-reduce-back-forth) which starts with higher values, but improves quickly outperforming the others before the 5000th model. (min-prefer-certain-locations) and (min-prefer-trucks) have slightly better results than the baseline, while (min-hubs-on-location) and (min-reduce-back-forth) have same or worse results (see table 14).

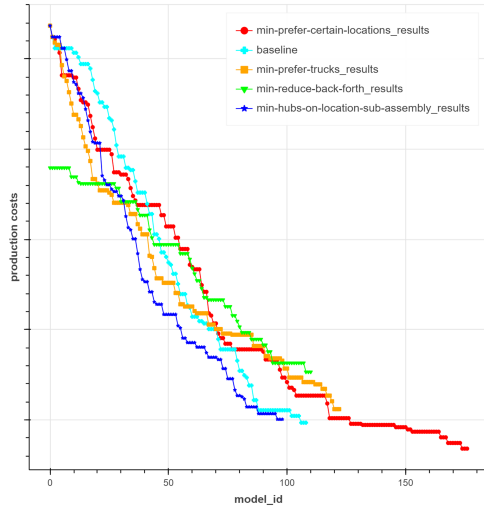
5.4.2 Variability

Jaccard similarities of the applied weak constraints move between 0.5 and 0.69 versus last (baseline) for the production and between 0.21 and 0.32 for the transportation.

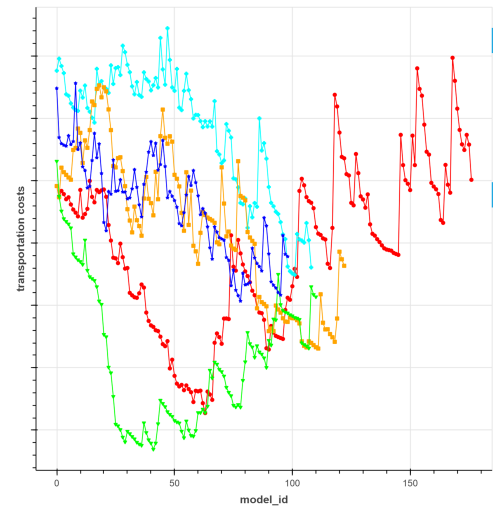
The similarities among the weak constraint configurations are 0.47 to 0.67 for production and 0.23 to 0.27 for transportation which are relatively narrow windows compared with the previous features, but confirming the tendency of higher variability of transportation compared with production. All figures can be seen in table 9.

5.4.3 Search Efficiency

Grounding took between 0.36 and 0.4 seconds for all configurations. Table 16 shows the percentage of models found versus the (baseline) run. As already mentioned in the KPI evolution analysis, (min-prefer-certain-locations)



(a) Evolution of the Production Costs for the tested weak constraints



(b) Evolution of the Transportation Costs for the tested weak constraints

Figure 21: Overview of the resulting KPIs from the weak constraint runs

Weak constraint	% of models cost to last (baseline)		
	Production	Transport	CO ₂
last (min-reduce-back-forth)	102%	93%	105%
last (min-prefer-certain-locations)	99%	121%	96%
last (min-prefer-trucks)	101%	100%	99%
last (min-hubs-on-location)	100%	103%	100%

Table 14: The KPI percentages of each weak constraint in relation to the last (**baseline**) model. The best results are highlighted in gray.

Heuristic	(min-red.-back-forth)		(min-hubs-on-loc.)		(min-prefer-certain-loc.)		(min-prefer-trucks)	
	p	t	p	t	p	t	p	t
(baseline)	0.5	0.21	0.69	0.32	0.69	0.26	0.63	0.27
(min-red.-back-forth)	1	1	0.5	0.25	0.5	0.23	0.47	0.23
(min-hubs-on-loc.)			1	1	0.67	0.27	0.59	0.28
(min-prefer-certain-loc.)					1	1	0.67	0.27
(min-prefer-trucks)							1	1

Table 15: Similarity comparison for all weak constraints, with respect to the production sites (p) and transportation means (t).

Weak constraint	number of models found after	
	optimum found	36.000sec
(min-reduce-back-forth)	no	83%
(min-hubs-on-location)	no	74%
(min-prefer-certain-locations)	no	132%
(min-prefer-trucks)	no	92%

Table 16: The number of models found for the different weak constraints

has significantly higher number of models found while the other weak constraints are significantly below the (baseline) efficiency. No optimum was found for any run.

5.5 Optimization statements

This section presents the result of the different optimization statement configurations.

5.5.1 KPI evolution

The evolution of the KPIs as visible in Figure 22 does strongly deviate from the results of the previous trials. The production cost evolution of the (**optim-1-2C-3T**) configuration is the only one with a similar behavior as the (baseline) and other configurations tested before ending up with 102% compared with it (see Table 17 for all values). The configuration (**optim-comb-Obj**) starts with lower values and is improving slowly, but there is no valid tendency to be observed as the number of models found is very low compared with all tested configurations and stops at 109% above (baseline). All other configurations move almost flat regarding the production cost finishing at 113% or 114% of (**baseline**) costs.

The Transportation Costs again evolve similarly with continuously improving values for all configurations prioritizing transportation or CO₂ costs ending up with significantly lower costs for both (see Table 17) compared to the (**baseline**). The (**optim-comb-Obj**) starts with the highest values, but shows a very steep improvement of values stopping at 109% with the timeout without showing hints of saturation. The (**optim-1P-2C-3T**) configuration show minor improvements with ups-and-downs finishing at 107% of the (**baseline**). The CO₂ costs show similar evolutions as the transportation cost, but behavior of all configurations is closer among each others. All configurations end up with better values than the (**baseline**) or at least equal, i.e. for (**optim-comb-Obj**).

5.5.2 Variability

The production similarities versus the (baseline) are between 0.2 and 0.25 for all configurations, except the (1P-2C-3T) configuration with 0.61. The transportation values versus (baseline) range between 0.14 and 0.21.

Among the optimization statements the values are between 0.21 and 0.65 for production and between 0.14 and 0.43 for transportation. Find all values in fig 18.

To visualize the impact of the alternative optimization statements on the KPIS as well as on the variability, see Figure 23. Again a *Pareto front* is implied, this time approached from different directions than previously observed.

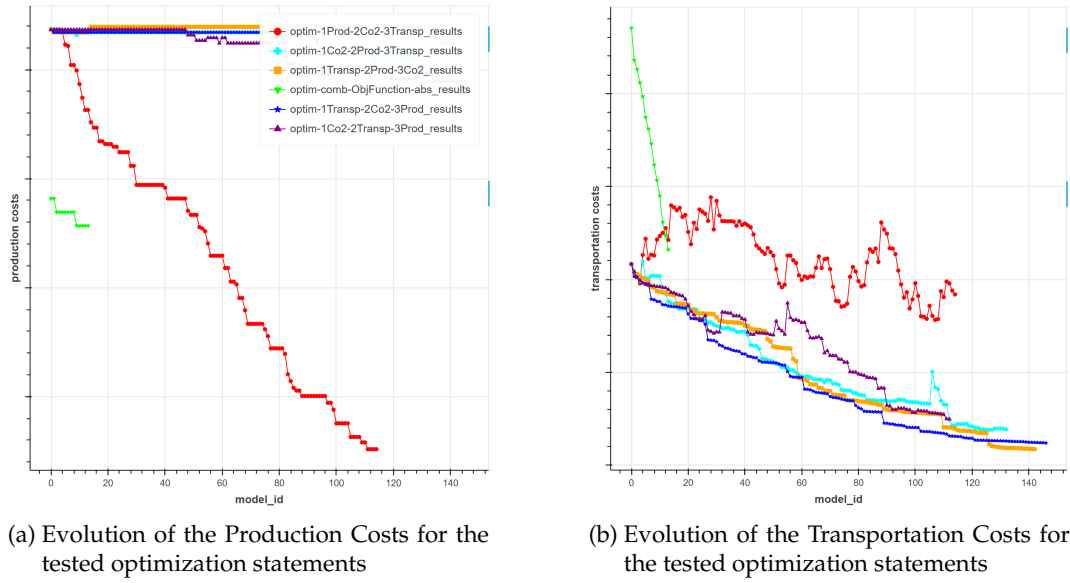


Figure 22: Overview of the resulting KPIs from the optimization statement runs

Optimization statement	% of models cost to last (baseline)		
	Production	Transport	CO ₂
last (1P-2C-3T)	102%	107%	84%
last (1T-2P-3C)	114%	44%	58%
last (1T-2C-3P)	113%	47%	62%
last (1C-2P-3T)	114%	52%	59%
last (1C-2T-3P)	113%	57%	60%
last (comb-Obj)	109%	125%	100%

Table 17: The KPI percentages of each optimization statement in relation to the last (**baseline**) model. The best results are highlighted in gray.

Heuristic	(optim-1P-2C-3T)		(optim-1T-2P-3C)		(optim-1T-2C-3P)		(optim-1C-2P-3T)		(optim-1C-2T-3P)		(optim-comb-Obj)	
	p	t	p	t	p	t	p	t	p	t	p	t
(baseline)	0.61	0.21	0.2	0.18	0.21	0.2	0.2	0.17	0.2	0.16	0.25	0.14
(optim-1P-2C-3T)	1	1	0.25	0.17	0.28	0.17	0.26	0.18	0.25	0.18	0.24	0.14
(optim-1T-2P-3C)			1	1	0.65	0.43	0.57	0.31	0.61	0.32	0.25	0.17
(optim-1T-2C-3P)					1	1	0.55	0.31	0.57	0.3	0.28	0.17
(optim-1C-2P-3T)							1	1	0.51	0.31	0.21	0.16
(optim-1C-2T-3P)									1	1	0.22	0.15
(optim-comb-Obj)											1	1

Table 18: Similarity comparison for all optimization statements with respect to the production sites (p) and transportation means (t).

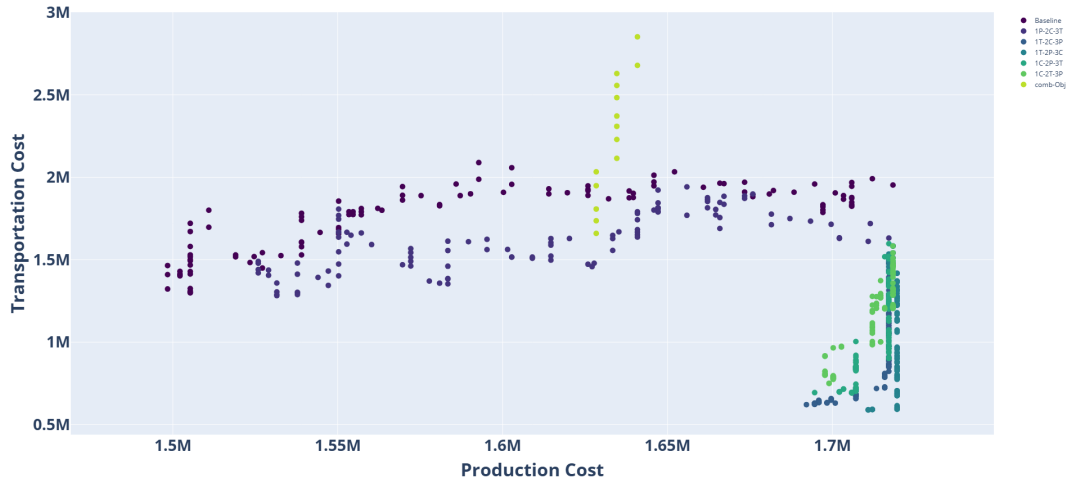


Figure 23: Evolution of the Production Costs with respect to the Transportation Costs for the tested optimization statements

5.5.3 Search Efficiency

Grounding too between 0.39 and 0.41 seconds for all configuration except for the (**optim-comb-Obj**), which took almost 5 minutes. Finally, the number of found models can be seen in tab 19. The only configuration with Production Costs as the top priority like the (baseline) configuration found 14% less models. The two configurations prioritizing the Transportation Costs are producing few more models than the (baseline) while the two prioritizing CO₂ produce few less. No optimum was found for any configuration.

5.6 Consolidated Result

To get the full and consolidated picture of all the tested features, all experimental results are gathered in one scatter plot and compared with trivial validation optimization runs. These validation runs were configured as described in section 4 and are supposed to provide the lowest values that can be achieved for production and Transportation Costs alone while at least holding the minimum constraints of feasibility. For the best achievable costs the rule (**workshare-IC**) from the (**baseline**) configuration was removed, so that solutions were only constrained by the facts (**pProduceableAt**). As a consequence, the resulting Production Costs reflect the best possible cost under realistic conditions, which is 1.442.125 (96% versus (**baseline**)). The best achievable transportation cost is 522.286 (40% versus (**baseline**)) accordingly.

See Figure 24 for a consolidated result with all tested features included. Every feature out of 11 is plotted with a dedicated color. Now the *Pareto front* is clearly visible. The distribution of the models across the scatter plot shows the deep penetration of the search space as only different models can produce different KPIs. The *Pareto front* is

Optimization statement	number of models found after	
	optimum found	36.000sec
(optim-1P-2C-3T)	no	86%
(optim-1T-2P-3C)	no	107%
(optim-1T-2C-3P)	no	110%
(optim-1C-2P-3T)	no	99%
(optim-1C-2T-3P)	no	84%
(optim-comb-Obj)	no	10%

Table 19: The number of models found for the different optimization statements

approached from diverse areas of the search space into different directions. The two lines mark the results of the best Production Costs (orange) and Transportation Costs (green) from the trivial configuration as described above. It shows that the best possible transportation cost is practically reached and the best production cost very close (note, that the scale of the production cost is much finer).

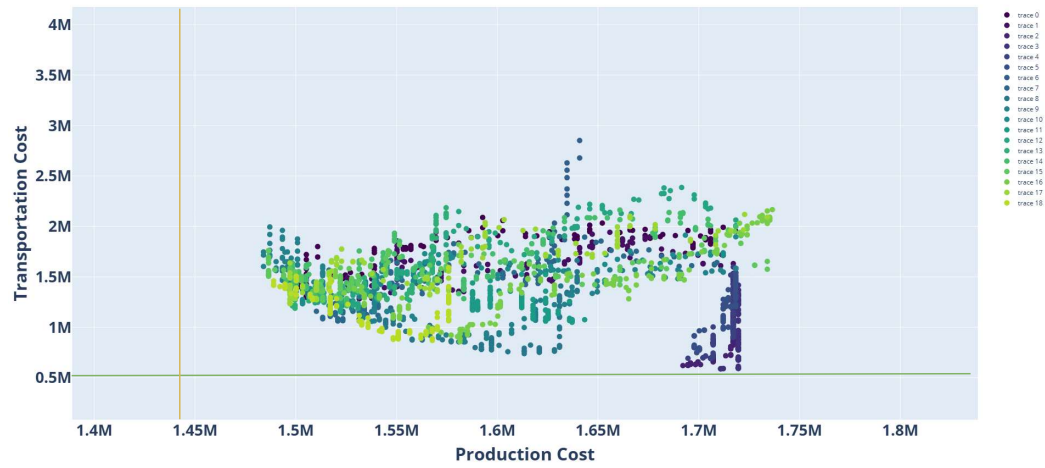


Figure 24: Evolution of the Production Costs with respect to the Transportation Costs for all tested features

6 Discussion of Main Phase

This section aims at interpreting the results presented in the previous section. It will compare the results with the hypotheses expressed in section 1.2 and the specific expected impacts of the features to be tested in section 3.3.

The first observation made is that, regarding the KPIs, the far biggest impact was achieved by modified weightings and prioritization of the KPIs as implemented with all optimization statements targeting transportation or CO₂ costs (see Table 17). The improvements of these two KPIs were between 40 to 56% for all tested optimization statements while improvements for production costs were rarely achieved by any tested feature or only 1% versus the **(baseline)** configuration which already prioritizes production costs. The consolidated scatter plot of the KPIs as in Figure 24 including the different KPI weightings reinforces the visibility of a *Pareto front* and demonstrates that with the use of different configurations the best achievable values for production and transportation costs are almost reached for both. This and the observation that the *Pareto front* is approached from different directions is providing an answer to the first research questions. The selection of the KPIs and the active weighting of the according optimization statements can lead to improved answer sets with regards to the objective functions, not only with better values for individual KPIs, but for many more balanced *Pareto optimal* solutions in between. A *Pareto front* is clearly indicated with higher variability of the KPIs and deeper penetration of the solution space. A higher variability of the answer sets is supporting this and will be described below.

The impact of the prioritization of the optimization statements seems to play a bigger role for all other means as well. This assumption is supported by looking at the KPIs evolutions for the tested features in Figures 16, 20 and 21. Regarding the production costs all tested features show a similar evolution over time with continuously improving values over time. The impact of the features is limited to accelerating or slowing down the improvements, but the general tendency remains. For the other KPIs, transportation and CO₂ costs the evolutions look rather random, even for those heuristics and weak constraints that were addressing them directly as described in section 3.4.1. Therefore the conclusions that can be drawn from the impact of the tested features on KPIs which are not prioritized could be limited while features that were targeting production costs, i.e. (*heu-prefer-certain-locations*) and (*min-prefer-certain-locations*) were leading to slightly better outcomes quicker as it was expected. These two rules also led to higher search efficiency reflected in more models found, especially for the weak constraint (+32%), while all other weak constraints found fewer models. This could indicate that weak constraints directly addressing the prioritized KPI can foster search efficiency.

The possible positive effect of weak constraints on search efficiency is also hinted by the tested rule (**nbSites-WC**). Again the weak constraint leads to significantly more models found (+26%, see Table 10) which hints to the assumption that the (**workshare**) constraint could be implemented more smoothly than with the (baseline) configuration and the according implementation as an integrity constraint, i.e. (*workshare-IC*). The (*minVal-IC*) still finds +13% more models. All together these observations can support the hypothesis that weak constraints can guide the solver more efficiently. Nevertheless it needs to be considered that KPIs of the different (*workshare*) configurations can not be compared directly as they modify the solution space differently.

Overall the rule (**heu-prefer-certain-locations**) had a positive impact on all KPIs while also slightly increasing search efficiency making it a good choice for general implementation. The weak constraint (**min-prefer-certain-locations**) had the best production costs from all features and increased search efficiency the most. **heu-hubs-on-location** produced better results for the *Pareto optimization* of production and transportation costs. The optimization statement (**optim-1T-2P-3C**) performed best for transportation and CO₂ costs. The workshare constraint implementation (**comb. minVal-IC/nbSites-WC**) produced good KPIs and still made solutions more desirable, because it incorporated important domain knowledge and is therefore recommended for all future runs.

Regarding research question 3, the results hint at positive impacts of the heuristics and weak constraints on search efficiency, especially when they target higher prioritized objective functions.

An observations worth mentioning are the results regarding the features concerning the optimization statements. As stated at the beginning of this section, only the prioritized KPI was showing the typical behavior of continuous improvement over time for all tested features, while transportation and CO₂ costs rather evolved randomly. This behavior changes for all optimization statements prioritizing transportation or CO₂ costs, making the values continuously improving. This emphasizes that it can make sense to apply active and alternative weighting of the optimization statements in order to be able to optimize as well for the other KPIs and to generate solutions that might dominate the solutions from the (baseline) configuration in the meaning of a *Pareto optimization* as described in section 1.2.3. It could be observed that search efficiency was highest when the optimization prioritization was on the KPI with the highest variability – both in terms of KPI values and solution candidates – which is transportation costs in this case. This might be a consequence of the solvers behavior when optimizing for multiple objective functions. Once it improves the value of the primary KPI, it will first optimize the lower priority KPIs before turning back to the highest priority and improve the value with one iteration. When the lower priority KPIs have significantly higher variability, this might be more time consuming for the overall optimization run. Further research is needed to investigate this hypothesis. The (*optim-comb-Obj*) configuration shows promising directions when looking at the KPI evolution curves (see Figure 21). Though, the search efficiency is low and the grounding took roughly five minutes compared to below 0.4 minutes for all other configurations.

It is clearly visible that all features that were tested were leading to diverse answer sets as reflected in the similarity

comparison Tables 9, 12, 15 and 18. Throughout all features similarities of the last models were between 0.14 and 0.43 for transportation and generally higher for production from 0.2 to 0.69. As a consequence, the variability of the production cost KPI was also low as described earlier. If a global optimum exists, this values could also indicate how close the solutions are to that with the experiments conducted so far with a timeout of 36000 seconds. In the case of a global optimum for production costs, the similarities need to reach 1.0 for different configurations. With all last models prioritizing the production costs, the average production similarity $\bar{J}(p)$ is relatively high, but still far from 1.0 with $\bar{J}(p) = 0.5$. This value incorporates a noticeable low production similarity of the (*heu-prefer-trucks*) configuration of only between 0.15 and 0.22 compared with the other heuristics applied. This might be the result of reducing feasible solutions to the assignment of parts to production sites when preferring truck transportation as not all sites can be connected via truck transportation. But the impact still seems higher than expected considering that still the (*workshare-IC*) rule from the (*baseline*) configuration requires all production sites to be involved in the logistics network.

The results presented earlier and the fact that production similarities average is 0.5 and transportation even below, imply that the timeout of 36000 seconds is not sufficient to converge to any optimum and to enable deeper conclusions about the features implemented. This is emphasized by the fact that no configuration found an optimum. It can also be concluded that the search space has still not been pruned sufficiently and further constraints or even facts need to be added to achieve convergence.

Some results are unexpected, such as the fact that the rules (*heu-hubs-on-locations*) and (*min-reduce-back-forth*) start with noticeable better solutions with regards to production costs despite the fact that these features target transportation cost optimization cost, so an adverse impact was rather expected.

Finally, the behavior of the combined objective function implemented with the rule (*optim-comb-Obj*) is remarkable (see Figure 22). It starts with the lowest production cost values and the transportation costs improve drastically. As the combined objective function requires an additional optimization statement which includes the sum of two terms that are subject to two optimization statements themselves, the grounding times are much higher and search efficiency gets very low due to the hugely increased number of ground rules (30 million vs. 1.5 million for all other configurations). Nevertheless, the fact that the evolution of the curves look promising and a combined objective function is a way to equally weigh two KPIs could lead to interesting results as it looks for more balanced *Pareto optimal* solutions than the weighted optimization statements which will look to find solutions on the *Pareto front* that are minimal with regards to the highest weighted KPI. The observation of the combined objective functions behavior seems promising on much longer run-times and guided search.

Summary regarding the research questions:

1. yes, the active and alternate weighting of the objective functions has a major impact on the results and does produce variability in the answer sets, increasing the solution space penetration and finding better solutions with regards to some specific KPI, but also finding more *Pareto optimal* solutions on the *Pareto front*. A combined objective function which weights different KPIs equally can be a good alternative for more balanced answer sets on the *Pareto front*.
2. yes with constraints: heuristics and weak constraints can help finding better solutions quicker, but only when they address the primary KPI. Otherwise their impact is weakened, but can still lead to much higher variability of answer sets during the search. The impact of heuristics and weak constraints on the overall outcome of objective functions seems moderate. But especially weak constraints seem to be able to increase search efficiency by guiding the search more smoothly than an integrity constraint could do.
3. yes, the combined use of all features can lead to better answers sets with regards to the objective functions and more variability of the solutions. The use of heuristics and weak constraints must be aligned with the weighting of the objective functions to find better solutions quicker.

Still, no configuration found an optimum, so the solution space was still not pruned sufficiently and the tested features were not able to provide enough guidance to converge to an optimum.

Following the results of phase 1 and the according discussion here, the following configurations will be tested with a 360.000 second timeout:

- (**conclusion-1**): targeting all KPIs, the best performing heuristic (**heu-prefer-certain-locations**) will be tested together with the generally implemented ones as below. the optimization statements remain as for the (**baseline**) configuration.
- (**conclusion-2**): targeting transportation and CO₂ costs, the best performing rules (**heu-hubs-on-location**) and the optimization statement (**optim-1T-2P-3C**) will be applied
- (**conclusion-3**): in order to increase search efficiency, the rule **min-prefer-certain-locations** will be applied together with the combined objective function (**optim-comb-Obj**) which looked promising with regards to more balanced KPIs.

All tested configurations will implement **(comb. minVal-IC/nbSites-WC)** into the baseline. The timeout will be extended to 360.000 seconds (10 times of previous experiments). These configurations follow the conclusion that the prioritization of the objective functions is a key driver of the runs' behavior and the impact of applied features is strongly dependent on it.

final configuration	% of models cost to last (baseline)		
	Production	Transport	CO ₂
last (conclusion-1)	97%	105%	182%
last (conclusion-2)	107%	52%	152%
last (conclusion-3)	108%	96%	143%

Table 20: The KPI percentages of each final configuration in relation to the last (**baseline**) model. The best results are highlighted in gray.

final configuration	(conclusion-1)		(conclusion-2)		(conclusion-3)	
	p	t	p	t	p	t
(baseline)	0.57	0.22	0.21	0.13	0.21	0.13
(heu-prefer-certain-locations)	0.65	0.2			0.21	0.14
(optim-1T-2P-3C)			0.16	0.14	0.22	0.14

Table 21: Similarity comparison for all final configurations, with respect to the production sites (p) and transportation means (t).

7 Results of Final Phase

7.0.1 Phase 2 Results

Taking the results from phase 1, the discussion of these and the according proposed configurations to be tested in a second phase, find now the results of the final configurations (**conclusion-1**), (**conclusion-2**) and (**conclusion-3**) as described at the end of section 6.

The (**conclusion-1**) configuration which intends to further improve the production costs KPI was able to find the best value so far with 97% versus (**baseline**) (see Table 20) and practically finding the best possible result holding the according constraints. This is supported by the evolution of the production costs over time, which implies a saturation. The transportation and Co2 costs are worse than for the (**baseline**) run though.

The last model of (**conclusion-2**) does not reach as good results for its main target transportation costs (-48%) compared with (**optim-1T-2P-3C**) (-56%) versus the (**baseline**), but is better for production costs making the configuration as good at least for a *Pareto optimization* of these two KPIs. Co2 costs are significantly worse. This configuration also shows first indications of a saturation as visible in Figure 25.

Finally, configuration (**conclusion-3**) with combined objective function remains above all reference (**baseline**) KPIs, but the values are better balanced across the KPIs and the evolution of the KPIs still looks promising as still no signs of saturation are shown for transportation costs.

7.0.2 KPI evolution

7.0.3 Variability

The similarities among the final configurations and their main inspired configurations show that they still differ strongly regarding their final models (see Table 21). Production similarity between (**min-prefer-certain-locations**) and (**conclusion-1**) is relatively high with 0.65, but still distant from 1.0.

7.0.4 Search Effectivity and Efficiency

The number of models found over time is steadily decreasing for all configurations. The solver seems to be struggling to find new, improved models especially for (**conclusion-2**), which found no better model anymore after the

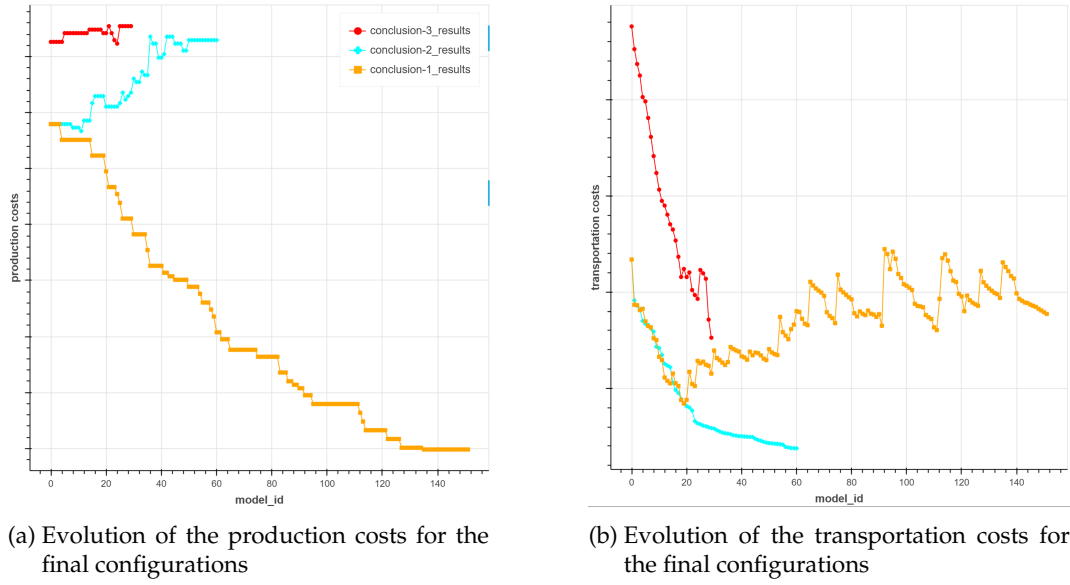


Figure 25: Overview of the resulting KPIs from the final configuration runs

final configuration	number of models found after	
	optimum found	360.000sec
(conclusion-1)	no	114%
(conclusion-2)	no	56%
(conclusion-3)	no	27%

Table 22: The number of models found for the different weak constraints

6.000th model which was found after less than half of the timeout. (**conclusion-3**) showed a heterogeneous behavior, generally still slow in finding models, but partially with better performance and phases of longer pauses. See Table 22 for details.

8 Discussion of Final Phase

In this section the outcome of the final configurations experiments will be discussed and whether they are able to confirm the expectations.

The **(conclusion-1)** configuration, which prioritizes the production costs, achieves almost the best possible values with 97% versus **(baseline)** configuration and only 1% above the best achievable value of the *(trivial-production)* configuration. On the other hand, the value for transportation costs is slightly higher (+5%) and CO₂ cost significantly higher (+82%). A similar effect is observable for the **(conclusion-2)** configuration for the prioritized transportation costs KPI and the others. The values are very close to the best achievable value of the **(trivial-transportation)** configuration taking into account that all configurations incorporated the rule **(comb. baseline/minVal-IC)**, which prunes the search space, but also removes possibly better solutions with regards to the KPI.

Two conclusions could be drawn from these figures: First, that with longer run-times the setup with several optimization statements and according weightings, the top-prioritized KPI will be able to achieve the best possible value. Second, that this will happen at the expense of the other KPIs, finally leading to more unbalanced KPIs, which are usually not the preferred solutions when going for a **Pareto optimization**. The **(conclusion-3)** configuration with the combined objective function seems to produce more balanced KPIs, but still has low search efficiency, possibly due to the huge number of ground rules as described earlier. A modified formulation of the combined optimization statement could be attempted to tackle this issue.

Still none of these configurations did find an optimum, but only showed signs of saturation close to the best possible values of the prioritized KPI for the **(conclusion-1)** and **(conclusion-2)** configurations. They were also struggling to find any new model a lot in advance of the timeout (see below). Apparently, the search space was still not sufficiently pruned, which is the main obstacle to converge to an optimum. Looking at the number of models found to get the best model, it took 15.100 models and 4.306 minutes for **(conclusion-1)** and 6.000 models and 1.180 minutes for **(conclusion-2)** to get there with the better overall results for *(conclusion-2)* (see Table 20). This reinforces the hypothesis about the possible positive impact of putting priority higher for the KPI that has higher variability.

9 Conclusion

9.1 Summary of Contributions

With the features tested in this thesis, it was possible to produce a clearly visible *Pareto front* for the primary KPIs production and transportation costs and hints of a *Pareto front* for all other combinations of KPIs. The best configurations achieved KPIs that were close to the best achievable values for production and transportation costs resulting from the trivial validation runs.

The tested heuristics have demonstrated that they can find better solutions quicker when they are targeting the prioritized KPI of the optimization run. As expected, solutions are getting closer to configurations without the use of heuristics on the mid and long term.

Similarly weak constraints could unfold their impact mainly when they addressed the KPI with the highest prioritization for the optimization run, either with slightly better results or often with a higher search efficiency.

The impact of heuristics or weak constraints were strongly weakened, invisible or even counterproductive when not addressing the KPI with the highest priority. All this demonstrates the importance of the optimization statements and the according weighting on the outcome.

This said, the results of the tested optimization statement configurations have the biggest impact on the KPI results and it is recommended to actively manage these to achieve the best results. Combined objective functions can lead to better balanced results of several KPIs than individual, prioritized optimization statements, which tend to optimize in favor of the highest prioritized KPI.

All tested features led to diverse answer sets. If there is an interest in exploring the solution space, the use of the tested features can help. Additionally the diverse solution space penetration offers better visibility of the *Pareto front* and provides more solutions on it which can be of interest.

None of the runs found an optimum, so the search space was still not pruned sufficiently and timeout times were still too low. Trivial test runs with single objective optimizations and simplified constraints provided lower bounds of the KPIs which showed that the obtained KPIs from the tested features were very close to the optima, but failed to converge.

There are some hints, that search efficiency can be improved when considering the variability of a KPI or the size of the search space that drives the according KPI. Putting higher priority to the KPI with higher variability could lead to shorter run-times.

Overall this thesis can contribute to the activities of industrial architects in reducing times to find good and diverse solutions on the *Pareto front* of two KPIs. It can support an iterative process to explore the solution space and find desirable configurations.

9.2 Future Work

The future work will tackle shortcomings and limitations of the current work, but also deal with additional capabilities that were not used for this thesis, either to ensure comparability with previous work, lacking maturity of the capabilities or due to resource and time restrictions.

No configuration and run has found an optimum, which necessitates future research to extend the timeout limit. Also, additional constraints and facts need to be added to further prune the solution space. For this, promising and desired parts of generated models can be used to set them as new facts or preferences, either as constraints or heuristics.

When testing additional features, especially such as heuristics and weak constraints, they should be tested together with optimization statements that prioritize the KPIs which is targeted most by the feature to be tested. This can help that the impact of the dedicated feature is not weakened. This does not only apply to new features, but also to some features tested in this thesis.

Some capabilities of *clingo* were not used for this thesis, but are planned to be implemented for further research. These include the optimization framework *asprin* and the solver *clasp*'s built-in option *parallel-solving*. *asprin* supports computing answers sets with preferences when optimizing with regards to some objective functions. One offered preference type is *pareto* and aims at finding *Pareto optimal* solutions as intended in this thesis. The option *parallel-solving* in *clasp* on the other hand enables multi-threading and can exploit the resources of the used 24-Core Processor hardware much more efficiently.

Finally, some work can be done on the user interface to provide helpful visualizations to the architect of the results so far, especially to browse solutions on the *Pareto front*. See an example of some PoC (Proof of Concept) implementations below.

References

- [1] E. Dietz, T. Philipp, G. Schramm, and A. Zindel, "A logic programming approach to global logistics in a co-design environment," *Electronic Proceedings in Theoretical Computer Science*, vol. 385, pp. 227–240, 2023.
- [2] J. R. R. A. Martins and S. A. Ning, *Engineering design optimization*. Cambridge and New York NY: Cambridge University Press, 2021.
- [3] W. Faber, G. Friedrich, M. Gebser, and M. Morak, eds., *Logics in Artificial Intelligence: 17th European Conference, JELIA 2021, Virtual Event, May 17–20, 2021, Proceedings*, vol. 12678 of *Lecture Notes in Artificial Intelligence*. Cham: Springer International Publishing and Imprint Springer, 1st ed. 2021 ed., 2021.
- [4] R. Arista, X. Zheng, J. Lu, and F. Mas, "An ontology-based engineering system to support aircraft manufacturing system design," *Journal of Manufacturing Systems*, vol. 68, pp. 270–288, 2023.
- [5] X. Zheng, X. Hu, R. Arista, J. Lu, J. Sorvari, J. Lentjes, F. Ubbis, and D. Kiritsis, "A semantic-driven tradespace framework to accelerate aircraft manufacturing system design," *Journal of Intelligent Manufacturing*, vol. 35, no. 1, pp. 175–198, 2024.
- [6] A. Falkner, G. Friedrich, K. Schekotihin, R. Taupe, and E. C. Teppan, "Industrial applications of answer set programming," *KI - Künstliche Intelligenz*, vol. 32, no. 2-3, pp. 165–176, 2018.
- [7] J. Sun, J. Tang, W. Fu, Z. Chen, and Y. Niu, "Construction of a multi-echelon supply chain complex network evolution model and robustness analysis of cascading failure," *Computers & Industrial Engineering*, vol. 144, p. 106457, 2020.
- [8] Q.-S. Hua, Y. Wang, D. Yu, and F. C. Lau, "Dynamic programming based algorithms for set multicover and multiset multicover problems," *Theoretical Computer Science*, vol. 411, no. 26-28, pp. 2467–2474, 2010.
- [9] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*. MIT Press, 3rd ed., 2009.
- [10] K. Thulasiraman and M. N. Swamy, *Graphs: theory and algorithms*. John Wiley & Sons, 2011.
- [11] W. Hochstättler and A. Schliep, *CATBox: An interactive course in combinatorial optimization*. Heidelberg: Springer, 2010.
- [12] Airbus, "Airbus digital twin." <https://www.airbus.com/en/newsroom/stories/2025-04-digital-twins-accelerating-aerospace-innovation-from-design-to-operations>, 2025. 20.05.2025.
- [13] Esra Erdem, Michael Gelfond, Nicola Leone, "Applications of answer set programming," *AI Magazine*, 2016.
- [14] M. Gebser, R. Kaminski, B. Kaufmann, and T. Schaub, "Multi-shot asp solving with clingo."
- [15] V. Lifschitz, *Answer Set Programming*. Springer eBooks Computer Science, Cham: Springer, 1st ed. 2019 ed., 2019.
- [16] J. W. Lloyd, *Foundations of Logic Programming*. Symbolic computation, Berlin and Heidelberg and New York and London and Paris and Tokyo: Springer, 2., extended ed. ed., 1987.
- [17] M. Gebser, B. Kaufmann, and T. Schaub, "Conflict-driven answer set solving: From theory to practice," *Artificial Intelligence*, vol. 187-188, pp. 52–89, 2012.
- [18] A. van Gelder, "Negation as failure using tight derivations for general logic programs," *The Journal of Logic Programming*, vol. 6, no. 1-2, pp. 109–133, 1989.
- [19] M. Gelfond and V. Lifschitz, "The stable model semantics for logic programming," 1988.
- [20] A. Ramos, P. van der Tak, and M. J. H. Heule, "Between restarts and backjumps," *Lecture Notes in Computer Science*, vol. 6695, pp. 216–229, 2011.
- [21] M. Davis and H. Putnam, "A computing procedure for quantification theory," *Journal of the ACM*, vol. 7, no. 3, pp. 201–215, 1960.

- [22] Martin Davis, George Logemann, Donald Loveland, M. Davis, G. Logemann, and D. Loveland, "A machine program for theorem-proving," *Communications of the ACM*, vol. 5, no. 7, pp. 394–397, 1961 // 1962.
- [23] J. P. Marques Silva and K. A. Sakallah, "Grasp-a new search algorithm for satisfiability," *Proceedings of International Conference on Computer Aided Design*; (1996) S. 220-227, pp. 220–227, 1996.
- [24] Steffen Holldobler, Norbert Manthey, Van Hau Nguyen, Julian Stecklina, and Peter Steinke, *2011 International Conference on Advanced Computer Science and Information Systems (ICACSIS 2011): Jakarta, Indonesia, 17 - 18 December 2011*; [proceedings. Piscataway, NJ: IEEE, 2011.
- [25] R. Comploi-Taupe, G. Friedrich, K. Schekotihin, and A. Weinzierl, "Domain-specific heuristics in answer set programming: A declarative non-monotonic approach," *Journal of Artificial Intelligence Research*, vol. 76, pp. 59–114, 2023.
- [26] University of Potsdam, "Potassco guide," 2019.
- [27] M. W. Moskewicz, C. F. Madigan, Y. Zhao, L. Zhang, and S. Malik, "Chaff," in *Proceedings of the 38th conference on Design automation - DAC '01* (J. Rabaey, ed.), (New York, New York, USA), pp. 530–535, ACM Press, 2001.
- [28] C. Baral, *Knowledge Representation, Reasoning and Declarative Problem Solving*. Cambridge University Press, 2003.
- [29] J. Rana, P. L. Gutierrez, and J. C. Oldroyd, "Quantitative methods," in *Global Encyclopedia of Public Administration, Public Policy, and Governance*, pp. 11202–11207, Springer Nature Switzerland AG, 2021.
- [30] G. Salton and M. J. McGill, *Introduction to modern information retrieval*. McGraw-Hill computer science series, New York: McGraw-Hill Book Comp, 1983.
- [31] A. J. Schmitt and M. Singh, "A quantitative analysis of disruption risk in a multi-echelon supply chain," *International Journal of Production Economics*, vol. 139, no. 1, pp. 22–32, 2012.
- [32] J. Rabaey, ed., *Proceedings of the 38th conference on Design automation - DAC '01*, (New York, New York, USA), ACM Press, 2001.
- [33] J. W. Fondahl, *A non-computer approach to the critical path method for the construction industry*. 1962.
- [34] M. Gelfond and V. Lifschitz, "Classical negation in logic programs and disjunctive databases," *New Generation Computing*, vol. 9, no. 3-4, pp. 365–385, 1991.
- [35] S. A. Cook, "The complexity of theorem-proving procedures," in *Proceedings of the Third Annual ACM Symposium on Theory of Computing (STOC)*, pp. 151–158, ACM, 1971.
- [36] L. Bertossi, W. Faber, B. Glimm, G. Gottlob, G. Ianni, D. Lembo, and S. Staab, eds., *Reasoning Web. Semantic Interoperability on the Web: 13th International Summer School 2017, London, UK, July 7-11, 2017, Tutorial Lectures*, vol. 10370 of *Information Systems and Applications, incl. Internet/Web, and HCI*. Cham: Springer International Publishing and Imprint: Springer, 1st ed. 2017 ed., 2017.
- [37] D. Abels, J. Jordi, M. Ostrowski, T. Schaub, A. Toletti, and P. Wanko, "Train scheduling with hybrid answer set programming," *Theory and Practice of Logic Programming*, vol. 21, no. 3, pp. 317–347, 2021.
- [38] Airbus, "Airbus production." <https://www.airbus.com/en/products-services/commercial-aircraft/the-life-cycle-of-an-aircraft/production>, 2025. 20.05.2025.
- [39] Airbus, "Airbus transportation fleet." <https://www.airbus.com/en/newsroom/stories/2023-10-building-a-lower-emission-maritime-transport-fleet>, 2025. 20.05.2025.
- [40] Airbus, "Airbus our worldwide presence." <https://www.airbus.com/en/about-us/our-worldwide-presence>, 2025. 20.05.2025.
- [41] Airbus, "Airbus design." <https://www.airbus.com/en/products-services/commercial-aircraft/the-life-cycle-of-an-aircraft/design>, 2025. 20.05.2025.
- [42] J. Sun, J. Tang, W. Fu, Z. Chen, and Y. Niu, "Construction of a multi-echelon supply chain complex network evolution model and robustness analysis of cascading failure," *Computers & Industrial Engineering*, vol. 144, p. 106457, 2020.